

A Web-Based Framework for Diabetes Prediction Using Machine Learning Approach

By
Alvy Rahman Niloy
0242220005101766

FINAL YEAR DESIGN PROJECT REPORT

This Report Presented in Partial Fulfillment of the Requirements for
the Degree of Bachelor of Science in Computer Science and
Engineering

Supervised by

Mr. Mahimul Islam Nadim
Senior Lecturer
Department of Computer Science and
Engineering Daffodil International University

Co-Supervised by

Mr. Golam Rabbany
Senior Lecturer
Department of Computer Science and
Engineering Daffodil International University



DAFFODIL INTERNATIONAL UNIVERSITY
Dhaka, Bangladesh

September 09, 2026

ACKNOWLEDGEMENTS

This work would not have been possible without the support and contributions of many individuals over the past two semesters. We are deeply grateful to everyone who has assisted us in one way or another.

First, we express our heartfelt thanks and gratefulness to the almighty for His divine blessing making it possible for us to complete the Final Year Design Project (FYDP) successfully.

We are grateful and wish our profound indebtedness to Mr. Mahimul Islam Nadim, Senior Lecturer, Department of Computer Science and Engineering, Daffodil International University, Dhaka, Bangladesh. Deep knowledge and keen interest of our supervisor in the field of Machine Learning to carry out this project. His endless patience, scholarly guidance, continual encouragement, constant and energetic supervision, constructive criticism, valuable advice, reading many inferior drafts, and correcting them at all stages have made it possible to complete this project.

We would like to express our heartfelt gratitude to the Head of the Department of Computer Science and Engineering, for his kind help in finishing our project and also to other faculty members and the staff of the Department of Computer Science and Engineering, Daffodil International University.

We would like to thank our entire course-mates at Daffodil International University, who took part in this discussion while completing the coursework.

Finally, we must acknowledge with due respect the constant support and patience of our parents.

ABSTRACT

The study aims to introduce a multi-model machine learning system designed for early diabetes prediction through web-based application, focusing on the aspects of accuracy, accessibility, and interpretability. The proposed system was developed and evaluated using the publicly accessible dataset of 100,000 patient records and eight clinical/demographic features. The classification algorithm models were trained and tested using 80%/20% stratified train-test split on five different models: Random Forest, Logistic Regression, XGBoost, LightGBM, and Decision Tree. LightGBM achieved the maximum accuracy of 97.24% with an F1-score of 81.04% which was the highest among the evaluated models, followed by XGBoost, Random Forest, Logistic Regression, and Decision Tree. The trained models were deployed in a complete web application with Flask based server-web framework with role based access for patients, doctors and administrators. The system provides multi-model risk forecasting, confidence ratings, risk classification based on the ensemble of models, prediction history, visualization of the performance, reports for download. The incorporation of SHAP-based explainability allowed it to reveal the contribution of each clinical feature to each prediction, making computations more transparent for healthcare workers and patients. The results show the accuracy and interpretability of machine learning models that can be applied to assess diabetes risk within a real-world web-based platform. But class imbalance, non-persistent data storage, and implementation of the solution in terms of security are limitations still to be addressed. Future work will aim at further development of data management, security, model robustness and clinical applicability for prospective implementation in real application.

Table of Contents

Approval	i
Declaration	ii
Acknowledgements	iii
Abstract	iv
List of Figures	vii
List of Tables	viii
1 Introduction	1
1.1 Introduction	1
1.2 Motivation	1
1.3 Objectives	1
1.4 Methodology	1
1.5 Project Outcome	1
1.6 Organization of the Report	1
2 Background	2
2.1 Introduction	2
2.2 Literature Review	2
2.2.1 Similar Applications	3
2.3 Gap Analysis	3
2.4 Summary	3
3 Research Methodology	4
3.1 Methodology/Requirement Analysis & Design Specification	4
3.1.1 Overview	4
3.1.2 Proposed Methodology/ System Design	4
3.1.3 Functional and Nonfunctional Requirements	5
3.1.4 Data Flow Diagram	5
3.1.5 UI Design	5
3.2 Detailed Methodology and Design	5
3.3 Project Plan	5
3.4 Task Allocation	5
3.5 Summary	5

4	Implementation and Results	6
4.1	Environment Setup	6
4.2	Testing and Evaluation/Performance/ Comparative Analysis	6
4.3	Results and Discussion	6
4.4	Summary	6
5	Engineering Standards and Design Challenges	7
5.1	Compliance with the Standards	7
5.1.1	Software Standards	7
5.1.2	Hardware Standards	7
5.1.3	Communication Standards	7
5.2	Impact on Society, Environment and Sustainability	7
5.2.1	Impact on Life	7
5.2.2	Impact on Society & Environment	7
5.2.3	Ethical Aspects	7
5.2.4	Sustainability Plan	7
5.3	Project Management and Financial Analysis	7
5.4	Complex Engineering Problem	8
5.4.1	Complex Problem Solving	8
5.4.2	Engineering Activities	8
5.5	Summary	8
6	Conclusion	10
6.1	Summary	10
6.2	Limitation	10
6.3	Future Work	10
	References	11

List of Figures

3.1 This is a sample diagram

4

List of Tables

2.1	Summary of Literature Reviewed.	2
5.1	Mapping with complex problem solving.	8
5.2	Mapping with knowledge Profile.	8
5.3	Mapping with complex engineering activities.	9

Chapter 1

Introduction

This chapter provides a foundation for the remainder of the report. It starts off introducing the reader to why the prediction of diabetes is important and the problem this project attempts to solve before introducing the motivation of why this project was chosen over more traditional diagnostic methods, which are based on machine-learning and a web deployed approach. It outlines the aims and agendas of the work, provides a brief summary of methodology used and finally ends with discussion about the goals to be found throughout the rest of the report and how it is structured.

1.1 Introduction

Diabetes mellitus condition is chronic metabolism disorder, in which the body does not produce sufficient insulin, or the insulin that it does produce is not used effectively, resulting in higher blood glucose and typically requiring life-long treatment. Over time if left untreated, it harms the heart, kidneys, eyes and nervous system, and is one of the world's most rapidly increasing non-communicable diseases. As per the latest global count by International Diabetes Federation, there were more than 537 million of diabetes patients in the world as of 2017 and is expected to reach over 780 million by 2045 if the current trend continues [1]. Bangladesh is no different, and perhaps even more alarming - because rates are on the rise here as a result of rising urbanization, shifting eating habits and declining physical activity, with screening access not being uniform even outside of cities.

A big proportion of individuals who are at risk of diabetes or who have it, do not receive an early diagnosis. Lab tests, such as a fasting glucose, or HbA1c level interpreted by a physician, is typically needed for diagnosis, but this path to diagnosis to treatment (clinic, test, doctor follow-up) is often too costly, takes too long or is not easily available in many areas of Bangladesh. The time between the onset of a viral infection and when the disease starts to multiply enough to show symptoms, is also when complications are more difficult to avoid. There could be a solution to part of this, if someone could be given a tool to check their risk early because they may already have information from a routine check-up, such as their age or BMI or blood glucose level and so on, and they become worried about it even before they're diagnosed.

This type of problem is ideal for machine learning, since diabetes risk cannot be simply explained by a single clinical or lifestyle factor, but needs to take into account the

interaction of multiple factors simultaneously. These interactions are difficult for traditional rule-based scoring systems to understand, but can be captured by machine learning-based classification models with sufficient numbers of medical records. This project follows this one straight away - it takes a set of 100,000 patient records and trains and compares a number of supervised classifiers: Random Forest, Logistic Regression, XGBoost, LightGBM and Decision Tree - it then makes the best-performing models available as a web app, enabling any user to obtain a risk estimate instantly and creates the same inference but without hiding in the background behind a black box.

1.2 Motivation

The idea behind the project arose from the fact that most of the published work on diabetes prediction in a paper merely reports an accuracy number, which then implies that that is all that accuracy consists of. Once the models are trained, evaluated, compared against a small number of baselines, and it ends there is a working system that a patient or a clinic will NEVER be able to implement. It's that disconnect between the "a model that performs well on a held out test set" and "a system that someone can operate" that this project sought to bridge. The hard part was not building the prediction pipeline, but getting it to something useful (and that the user could use, not just the pipeline authors).

Note also that there is a more computational rationale for the design decisions taken here. The use of a single algorithm implies depending on specific assumptions for this algorithm, a linear algorithm such as Logistic Regression is easier to interpret but may not capture non-linear relationships between the features like BMI/HbA1c while a tree ensemble type model like a Random Forest, XGBoost or LightGBM can capture those relationships but may be not as easy to interpret individually. Putting in several models side by side and showing all of their results (with a confidence measure), is better than taking one number from one algorithm and assuming it was correct. This comparative method additionally allowed us to investigate, in a quite concrete manner, the accuracy-interpretability balance that is discussed in many healthcare-related application papers during the machine learning literature.

To go along with all this, this project has allowed us to experience a full, realistic machine-learning deployment process, which ultimately involved us cleaning a messy real-world set of data, comparing various models, creating an app, as well as an API and deployment security. That end-to-end experience - from the debugging of various things such as session handling, feature order mismatch or tunneling instability - is just as valuable to us as the final accuracy numbers are, and probably more.

1.3 Objectives

The ultimate goal of this project is to create a model that becomes accurate, interpretable, and deployable as a web-based system to predict the risk of diabetes, instead of a model created in a research notebook. The following specific objectives are translated to this aim:

- To preprocess and prepare a large diabetes dataset containing 100,000 patient records and eight clinical and demographic features for reliable machine learning analysis.
- To train and compare five supervised classification models - Random Forest, Logistic Regression, XGBoost, LightGBM, and Decision Tree under consistent experimental conditions.
- To evaluate and identify the best-performing model using accuracy, precision, recall, F1-score, confusion matrix, and other relevant performance metrics.
- To develop a Flask-based web application that provides real-time diabetes risk predictions using multiple trained machine learning models with confidence scores.
- To implement separate patient, doctor, and administrator portals for prediction, monitoring, data visualization, and system management.

1.4 Methodology

The methodology utilized was a relatively typical applied machine-learning process aimed at culmination in a deployed application, as opposed to a static report. The first step was to download the “Diabetes Prediction Dataset” from Kaggle, which contains 8 input variables: gender, age, hypertension, heart_disease, smoking_status, BMI, HbA1c, blood glucose level and a target variable, diabetes, which is a binary flag indicating whether a person has diabetes or not. Missing numbers were completed with the median number in each column of the data, missing categorical data was imputed using the mode value and then the two categorical features (gender, smoking history) were label encoded after which all the features were normalized using a StandardScaler.

After the data was in a usable form it was analysed using Exploratory Data Analysis (EDA) to interpret the balance of classes, correlation between features and associations between the independent variables (BMI, glucose, smoking history) and the outcome variable (diabetes). The cleaned and scaled data was later divided into training and test sets in a stratified fashion in the ratio 80:20 and five classifiers namely Random Forest, Logistic Regression, XGBoost, LightGBM, and Decision Tree were trained with the same set of parameters. They were compared by means of accuracy, precision, recall, F1 score and confusion matrices against a held-out test set of each model, which was done in

order to find the best possible model.

The models trained, preprocessing objects (scaler and encoders) were then serialized and integrated into a Flask web-app based application following the Model-View-Controller model. The application comprises a patient portal to input the patient's clinical values, a multi-model prediction interface showing the patient multi-model predictions with confidence scores and explanations using Shapley contributions, and separate patient doctor and administrator interest groups for monitoring the performance of the patient's interest groups, accessing patient prediction logs, inspecting the patient data. The entire app was deployed using a web server, the Ngrok tunneling service, in the Google Colab environment and then tested via a normal browser, rather than just the Google Colab environment.

1.5 Project Outcome

- **ML Performance:** All five models performed well, with LightGBM achieving the best performance, followed by XGBoost and Random Forest.
- **Model Comparison:** The results guided which models and predictions were presented to users through the web interface.
- **Working Web Application:** Developed a multi-role system where patients can enter clinical data and receive predictions from all five models.
- **Patient Features:** Includes ensemble risk score, simple recommendations, SHAP explanations, and downloadable PDF reports.
- **Doctor & Admin Dashboards:** Provide model performance, charts, and prediction records.
- **Explainability:** SHAP identifies the key factors influencing each prediction rather than only showing high/low risk.
- **Limitations:** Class imbalance, non-persistent patient records, and authentication security require improvement before real-world deployment.

1.6 Organization of the Report

The remainder of this report is divided into six chapters which trace the development of the project from background conception to technical results and general implications.

Chapter 1: Introduction

Chapter 1 introduces the project, presenting the background, problem statement, objectives, scope, significance, and overall motivation for developing a web-based diabetes prediction system. It establishes the context of the study and outlines the key challenges addressed by the proposed framework.

Chapter 2: Background

Chapter 2 is the background and literature review parts of this project which places this project in the grand scheme of predictive machine learning for diabetes. It introduces some of the studies that have been conducted recently, provides a summary of the models and reported accuracy of these studies and then identifies the individualized gaps, which included multi-model deployment, role based access, and explainability, that have been addressed by this project.

Chapter 3: Research Methodology

The methods employed in the research work are detailed in Chapter 3. It covers the nature of the data set and its features, the data processing/making features, rationale for the selection of the five classification algorithms, and design of the overall system and data flow from the web application and the design of the patient, doctor and administration portals.

Chapter 4: Implementation and Results

The implementation and results are given in Chapter 4. It outlines the experimental setup and presents the results of the performance of the various models produced on the held-out test set, along with confusion matrices, comparison charts and explanations derived from the use of SHAP. It will also highlight the problems faced in implementation and practical strategies used and applied for rectification like class imbalance, session/deployment instability and how they have been solved.

Chapter 5: Engineering Standards and Design Challenges

Chapter 5 looks at their engineering standards and design challenges, including software standards that were followed, hardware standards, and communication standards, as well as the design's societal, ethical, and sustainable aspects, and then examines the project in relation to the complex engineering problem-solving criteria.

Chapter 6: Conclusion

Finally, chapter 6 summarizes the results obtained, discusses the current limitations of the project, and proposes specific directions for further studies, such as looking into class-imbalance correction, the persistence of the database, and more secure ways of deploying the project before it reaches the clinic.

Chapter 2

Background

This chapter provides an introduction to the conceptual and research context of this project. This project begins by providing background on how to understand the following parts of the report, then gives an overview of the previous literature on diabetes prediction using machine learning, considers existing systems of diabetes prediction in the real world of the web and mobile (wireless) platform, and concludes by stating what gaps it seeks to rectify.

2.1 Introduction

In fact, predicting diabetes with machine learning is not a novel phenomenon; over the past decade or more people have been using classification algorithms with datasets such as the Pima Indians Diabetes dataset. More recently the amount of data that is used, and the sophistication of the algorithms applied to the data, have changed. The large amounts of data have become a huge advantage for ensemble and boosting models (e.g., Random Forest, XGBoost, LightGBM) compared to linear models, which increasingly assist in getting the most out of these newer and much larger datasets with tens or hundreds of thousands of records.

Concurrently, a common hurdle in many of these studies seems to be that a good model that was trained on a notebook isn't necessarily going to be something that a patient or a clinician can use. The typical published works are short and report accuracy, precision, and recall - in addition to F1-score - on a held-out test set, and that is it. It is hard to find anyone willing to go the extra mile, and wrap that model in a meaningful interface, let alone to isolate access to that model, by role (patient versus clinician versus administrator), let alone to explain, rather than just assert, why a given prediction was made. This project lies at the intersection of these three concerns (predictive performance, deployability, and interpretability) and the background review in this chapter focuses on understanding how well (or poorly) the current work has dealt with these concerns.

2.2 Literature Review

The problems studied to predict diabetes from various perspectives include: dataset size and quality, model selection, data class imbalance issue, explainability of models, and rarely deployment. The following review is based on papers published between 2023 and 2026, which directly informed the design of the features and capability for a deployable system beyond raw accuracy for this project, specifically model selection.

Sun et al. [2] compared six machine learning models on 556,495 medical records and found XGBoost to perform best with 97.5% accuracy. However, their work focused only on modelling without deployment or role-based features. Hasan et al. [3] combined AutoML with explainability techniques such as SHAP and LIME and developed a Streamlit application. However, the small Pima dataset and lack of patient-clinician role separation limited its practical use. The BRFSS study highlighted class imbalance as a major challenge in diabetes prediction, particularly affecting recall for the diabetic class. This provided a useful background for selecting evaluation metrics in this project.

Das et al. [5] compared several classical classifiers, including SVM, Random Forest, Decision Tree, Logistic Regression, and Gaussian Naive Bayes. Their main contribution was a direct methodological comparison, although no deployment layer was developed. Narendra and Shivaprashad [6] developed a functional web application using multiple models, with LightGBM achieving the best accuracy of 88.46%. However, the system lacked role-based access and real-time prediction logging.

Rustam et al. [7] focused on feature engineering and demonstrated that improved features can enhance prediction accuracy. However, their work remained at the prototype level without deployment. Islam et al. [8] developed a Web and Android-based diabetes prediction system for Bangladeshi females using XGBoost, ADASYN, and SHAP/LIME. However, it focused on a specific patient group and used a single-model approach. Chowdhury et al. [9] demonstrated that SMOTE can improve diabetic class prediction in imbalanced BRFSS data, making it relevant as a potential future improvement for this project.

Amara et al. [10] combined SMOTE, Random Forest, and SHAP, showing that class balancing and explainability can effectively work together. Kiran et al. [11] reviewed 33 years of AI and machine learning research in Type 2 diabetes prediction, highlighting the shift toward advanced models and explainable AI. A comparison between SHAP and LIME [12] found SHAP to provide more consistent feature explanations, which inspired its use in this project's explainability layer. Hasan et al. [13] and Manjrekar et al. [14] explored federated learning and ensemble learning approaches, respectively, representing potential future directions for diabetes prediction research.

Table 2.1: Summary of Literature Reviewed.

Author(s)	Year	Model(s) Used	Limitation	Accuracy
Sun et al.[2]	2025	XGBoost, RF, LR, DT, CatBoost, LightGBM	No web deployment; no multi-model UI; no RBAC	97.5%
Hasan et al.[3]	2025	AutoML + SHAP/LIME/LIME/IG	PIDD only (768 samples); no RBAC; no confidence score	85.01%
Ekambaram et al.[4]	2025	RF, XGBoost, GBM, DT, CNN (BRFSS)	No web deployment; class imbalance unresolved; no RBAC	~92%+
Das et al.[5]	2025	SVM, RF, DT, LR, Naive Bayes	PIDD only; no deployment; no confidence scoring	~90%+
Narendra & Shivaprasad [6]	2024	RF, SVM, LightGBM, KNN (Web App)	Single model deployed; no RBAC; no real-time log	88.46%
Rustam et al. [7]	2024	Ensemble + feature engineering (Kaggle)	No web interface; no RBAC; offline only	~95%+
Islam et al. [8]	2023	XGBoost+ADASYN+SHAP/LIME (Web+App)	Female-only dataset; single model; no RBAC; no admin panel	~81%

Chowdhury et al. [9]	2024	RF, GBM + SMOTE (BRFSS)	No deployment; no confidence scoring; no web UI	~90%+
Amara et al. [10]	2025	RF + SMOTE + SHAP (interpretable)	1,500-record subset only; no full-scale deployment; no RBAC	96.91%

2.2.1 Similar Applications

It's not just the strictly academic literature, though, that it is useful to look at the stuff already out there including research prototypes as well as actual, commercially available consumer products that try to do something similar to what this project is doing, because a prediction system on the web is being evaluated against what already exists in the space, not with an abstract.

- The most similar prototype to this project is Narendra and Shivaprasad's web application [6], which uses patient data to generate predictions with a trained model. However, it is limited to a single model and lacks role-based access for patients and clinicians.
- Islam et al.'s Web and Android-based system [8] is the most contextually relevant comparison because it focuses on Bangladeshi patients. However, it targets only female patients, uses a single-model approach, and lacks administrative oversight.
- Hasan et al. [3] demonstrated an easy-to-use explainable AI interface using interactive inputs and SHAP/LIME visualizations. Although designed mainly as a research demo, its presentation approach influenced the SHAP-based explainability features of this project.
- The CDC Prediabetes Risk Test represents a simpler rule-based screening approach, while consumer applications such as mySugr focus mainly on diabetes management after diagnosis rather than early risk prediction.

These four examples reveal a consistent theme actual deployed systems in this space are often very narrowly scoped: they are limited in the kind of demographers they allegedly serve; they concern a few dozen models, at best, that are made available; they are hard to explain; or, they are only intended for patients who have already been diagnosed with health problems, not for at-risk patients. The project's role is to less the

generation of a new algorithm, and more to synthesize what each of the above systems is separately capable of doing, in this case multi-model prediction, explainability, role-based access, and a pre-diagnosis screening focus into an all-encompassing system that can do it all easily and in a way that is coherent.

2.3 Gap Analysis

Reviewing the above research studies and deployed systems together, certain common gaps are seen. Nearly all studies with good accuracy do not make it past the modeling phase to become an actual tool. Of the very few that are deployed, none are pairing two or more models in a single GUI, no one is separating patient access from clinician/administrative access, and very few are providing patient prediction along with a true explanation of how the prediction was achieved, as opposed to simply a probability score. The table below makes the above comparisons feature by feature with the systems of greatest relevance to this project and what these systems have actually been implemented here.

Table 2.3: Gap Analysis Summary

References	Sun et al. (2025)	Hasan et al. (2025)	Ekambaram et al. (2025)	Das et al. (2025)	Narendra et al. (2024)	Rustam et al. (2024)	Islam et al. (2023)	howdhury al. (2024)	Amara et al. (2025)	This Study 2026)
Large Dataset (100k+)	Yes	No	Yes	No	No	Yes	No	No	No	Yes
Multi-Model Framework	Yes	No	Yes	Yes	No	No	No	No	No	Yes
Web Deployment	No	Yes	No	No	Yes	No	Yes	No	No	Yes
RBAC /Dual Role	No	No	No	No	No	No	No	No	No	Yes
Doctor-facing dashboard	No	No	No	No	No	Yes	No	No	No	Yes
Admin dashboard with live analytics	No	No	No	No	No	No	No	No	No	Yes
Explainable AI (SHAP/LIME)	No	Yes	Yes	No	No	Yes	No	No	Yes	Yes
Prediction history	No	No	No	No	Yes	Yes	No	No	No	Yes

2.4 Summary

This chapter analyzed the existing human health prediction technologies via machine learning approaches which were implemented in the following twelve studies published from 2023 to 2026 and five systems for human health prediction found on the internet (web/mobile) that have some elements that take on similar attempts. On the modeling front, steady progress has also been demonstrated, with datasets getting larger by the day, and ensemble and boosting methods like XGBoost or LightGBM clearly outperforming simpler linear models, and increasingly doing so in conjunction with explanations of the model, rather than as add-on strategies. In both, we see a severe lack of integration - training multiple models in a single system and then comparing their results as a whole; having patient access independent of clinician oversight; maintaining active prediction scores; and providing the people using the system the explanation behind the prediction - all in one deployable application, fairly consistently. In fact, this chapter indicates that none of the systems reviewed today do all these tasks simultaneously and in fact, the methodology described in the next chapter was devised to bridge that gap.

Chapter 3

Research Methodology

This chapter is a walkthrough of the construction of the diabetes prediction system from the initial requirement analysis, to the final system design. It begins with an overall approach, moves on to functional requirements and non-functional, and then presents a data flow diagram depicting how data flow throughout the app while also running through the user interface design before finishing with the project plan and the allocation of tasks throughout the week.

3.1 Requirement Analysis & Design Specification

3.1.1 Overview

This project is not just a research project, it is a systems building project, and the design questions began with a much more pragmatic one: what needs does a person have from a diabetes screening tool, and what does the clinician or administrator who deals with the screening tool need in return? The answer involved a three role architecture from the outset, having a patient interface role that patients use to provide their health information to the system and receive predictions from it, a doctor interface role to see the predictions for all patients, and an administrator role to monitor the system in terms of registered users, prediction performance on the system, and view logs of predictions made by the system.

The overall design style is inspired by the Knowledge Discovery in Databases (KDD) process; this particular process is a logical match for a project that goes from data to working interface. There were six broad stages in the design: acquisition and understanding of the dataset, preprocessing and feature engineering, exploratory data analysis, training and piloting of several models, development of web application based on best models, and finally, deployment and end to end (e2e) testing of the system. The choices in each phase were directly influenced by the choices made in the previous phase - choices for preprocessing were influenced by what the EDA discovered with the data as regards the problem of class imbalance, choices of model were influenced by the results produced by the preprocessing, and choices for the web application were influenced by what could be shown by the trained models within realistic real-time constraints.

3.1.2 System Design

The system is not based on a single classifier, but a multiple modeling prediction engine based on the concept of comparison. To train an ensemble of the algorithms, five supervised learning algorithms have been trained side by side on the same set of data, where only the best five algorithms chosen to ensure resistance to overfitting and clean probabilistic output of each algorithm. To train an ensemble of the algorithms, five supervised learning algorithms have been trained side by side on the same set of data, with some of the best five algorithms chosen to ensure resistance to overfitting and clean probabilistic output of each algorithm. Instead of eliminating each one of the other "losers" and retaining only one "winner," all five models are retained for prediction time: When a patient enters into a predictive system, all five models will be active and produce their own predicted values and confidence, and the suggested best model (based on the best measured accuracy) will be selected and highlighted.

The application uses MVC as its architectural pattern. Flask performs as a controller which serves RESTful routes, handling requests and returning either rendered HTML or JSON. The view layer is provided in HTML, rendered in Python (render_template_string); the whole application resides in a single script so no separate templates directory is necessary. The trained classifiers, the scaler and label encoders are part of the model layer.

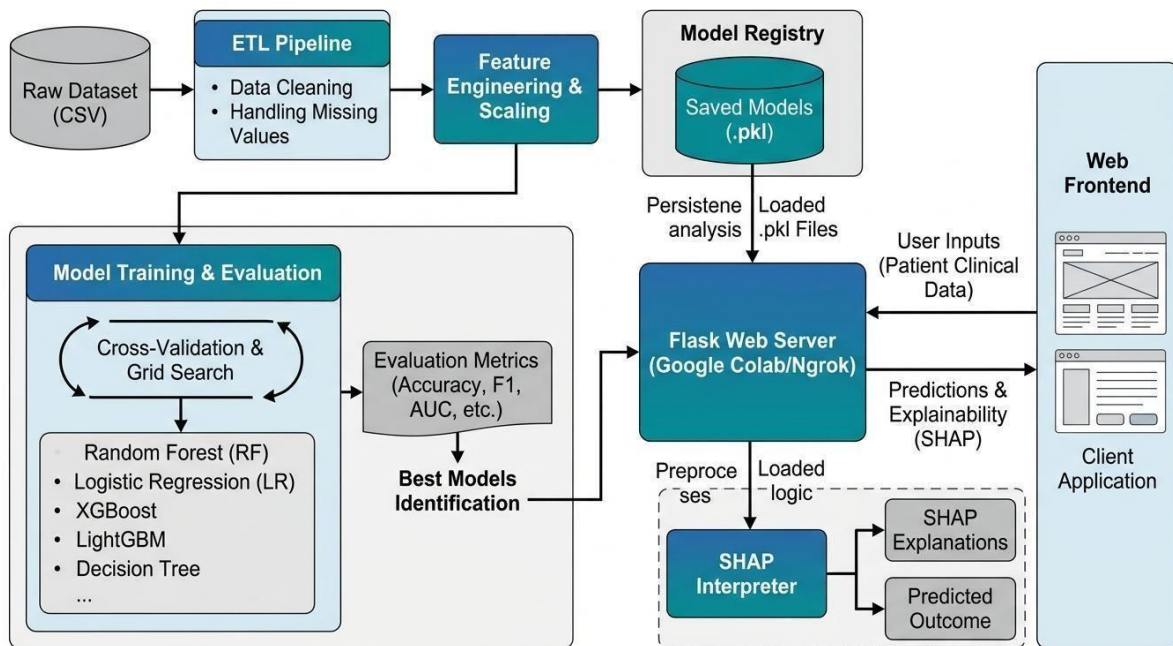


Figure 3.1: System Design diagram

3.1.3 Functional and Nonfunctional Requirements

Functional requirements define the concrete features, behaviors, and capabilities the system must actively provide to its three core user groups: patients, doctors, and administrators. Nonfunctional requirements establish the critical quality attributes, performance constraints, and security standards necessary to deliver a reliable, scalable, and trustworthy medical tool.

Functional Requirements

The functional requirements are statements of the actual system functions - what the patient/doctor/administrator can accomplish with the system - the concrete features the system can provide.

- **Patient registration and login:** A patient may register and log in to his personal patient page with the name of the patient, age, gender, email and the contact phone number.
- **Risk prediction:** Given a logged-in patient, enter 8 clinical values (gender, age, BMI, HbA1C level, blood glucose level, smoking status, hypertension, heart disease), and all 5 trained models provide an instant prediction.
- **Confidence scoring:** Not only does each model return a class-“Diabetic”/“Non-Diabetic” label, but a random number (also called a “confidence score”) indicating how strongly the model leans toward its prediction, allowing the patient to see the extent to which the model is misclassifying his/her data.
- **Ensemble risk summary:** The ensemble of five models uses the average risk probability computed across the models, and outputs a Low, Moderate or High level of risk, and a plain language recommendation.
- **Explainability:** SHAP values are calculated for each of the predictions and reflected with the patient, highlighting which of the provider's input features were most influential.
- **Downloadable report:** This summary of a patient's risk assessment can be downloaded by them as a pdf file, which they can keep or share with a doctor.
- **History of prediction:** Every prediction made by a patient is recorded into their history, which can be seen through the Patient's Dashboard.
- **Doctor dashboard:** Registering doctors can check out overall statistics, the performance of models, exploratory data charts and predictions made by patients throughout the platform.
- **Admin dashboard:** Administrator may login and get system-wide statistics, manage and inspect registered users (patients and doctors), preview and get

a full log of predictions and also view the underlying system data.

Nonfunctional Requirements

Non-functional requirements are quality attributes that the system must possess to be usable, which aren't specifically mentioned as features.

- **Accuracy:** Avoid a high 'overall' accuracy that excludes a poor performance on the minority class by under-performing on that class, but still maintains a high and balanced accuracy across both classes.
- **User-friendliness:** A patient is supposed to be able to complete a prediction in less than two minutes, even without technical literacy.
- **Security:** Doctor and admin dashboards should be secured using authentication and passwords shouldn't be kept in plain text.
- **Availability:** During a demonstration or testing session, the system should be available online to public URL throughout the entire session.
- **Scalability:** the architecture should enable adding further models or further clinical features in the future without the need for a redesigned system.
- **Maintainability:** Code base should be well organised to allow a model, route or dashboard tab to be added without disrupting anything.
- **Transparency:** They never need to be reported as simply a label; there should always be some sort of explanation or confidence level for the prediction, since this is a health-related tool.

3.1.4 DataFlow Diagram

Data passes the system in a fairly linear fashion, from raw in to user's result out. The proposed system is shown on Figure 3.1.4 (data flow diagram)

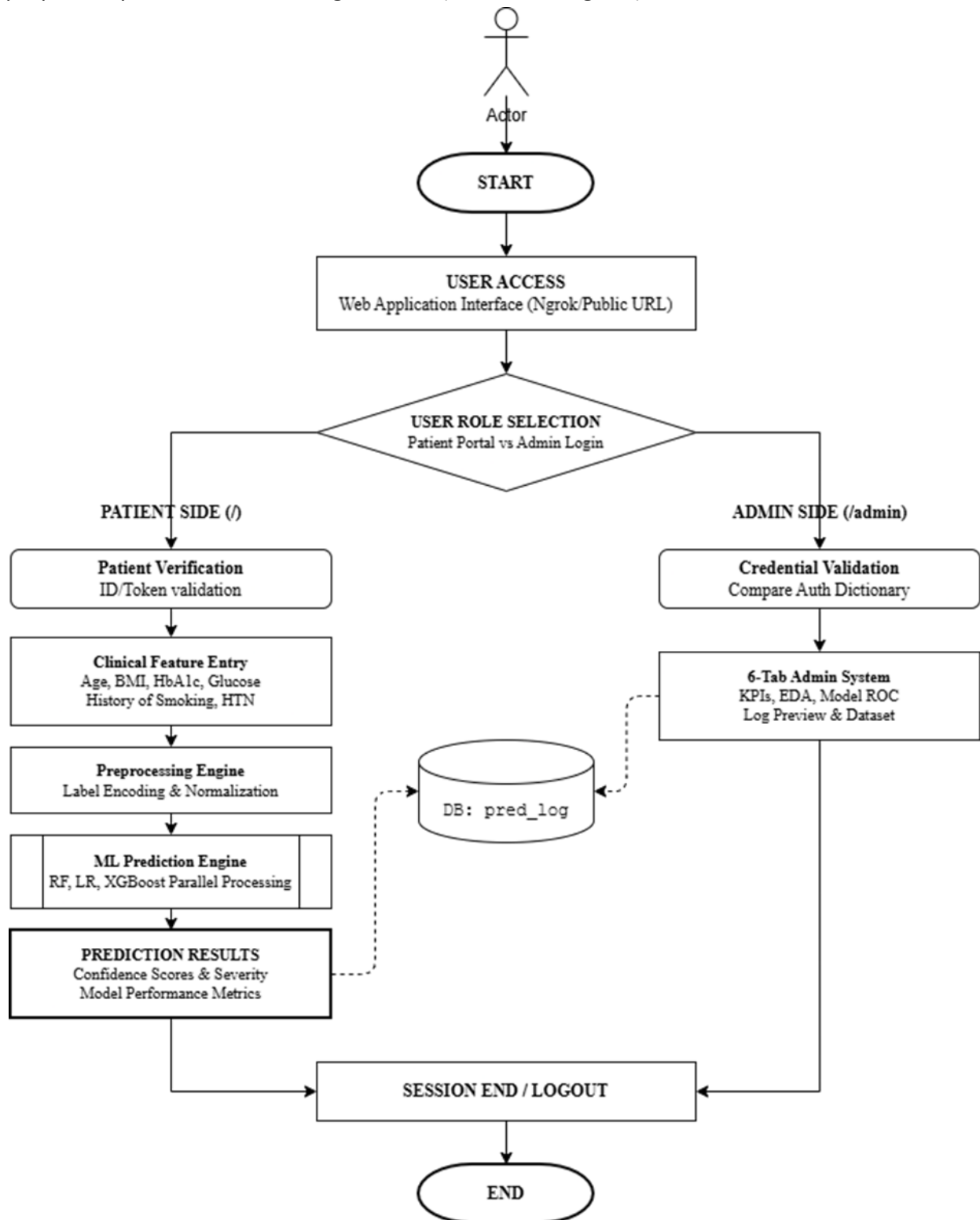


Fig. 3.1.4: System Prototype Flowchart - Patient Portal and Admin Dashboard Workflow

3.1.5 UI Design

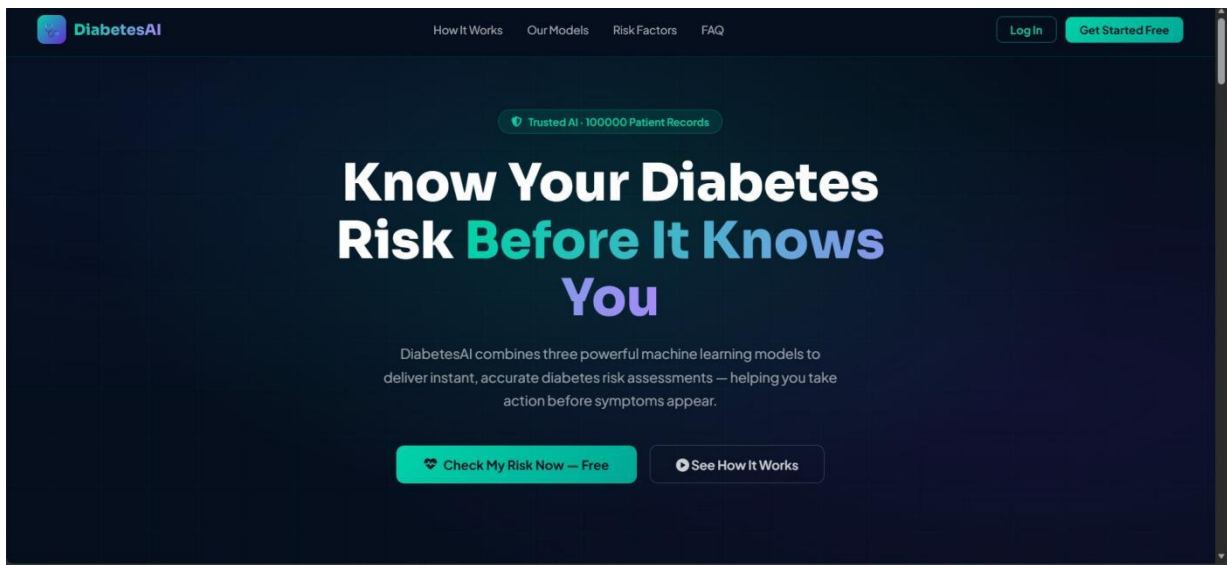


Figure 3.1.5: Landing Page of Web Interface

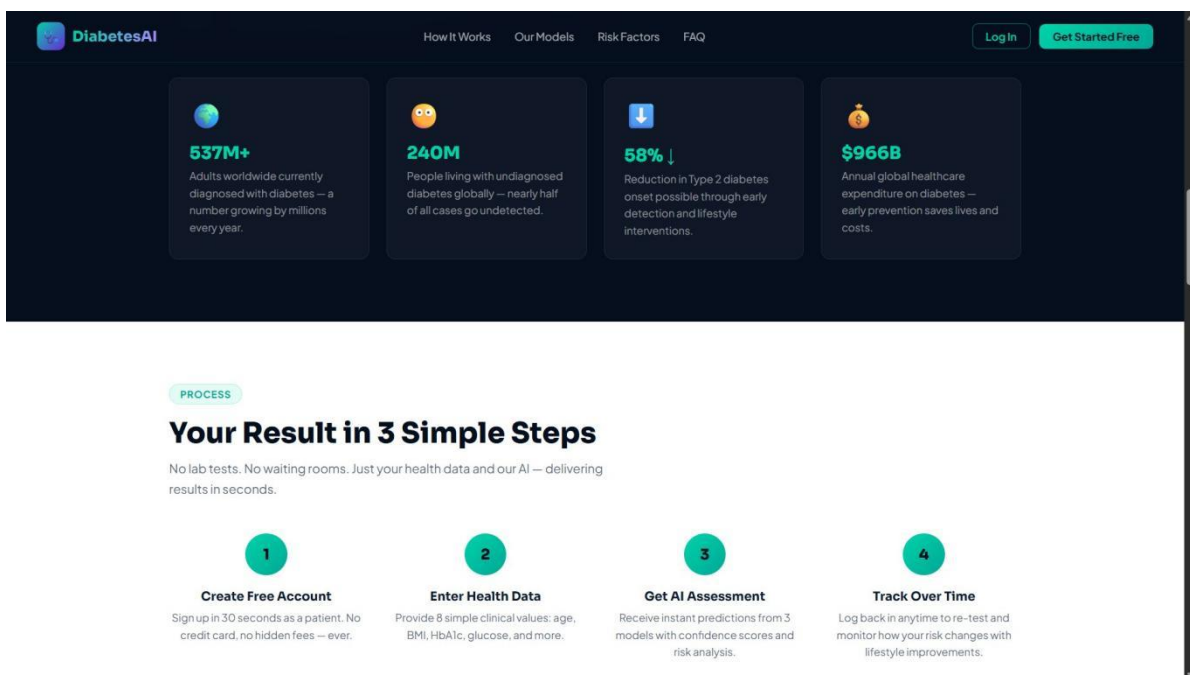


Figure 3.1.5: Home Page (Guidelines for User)

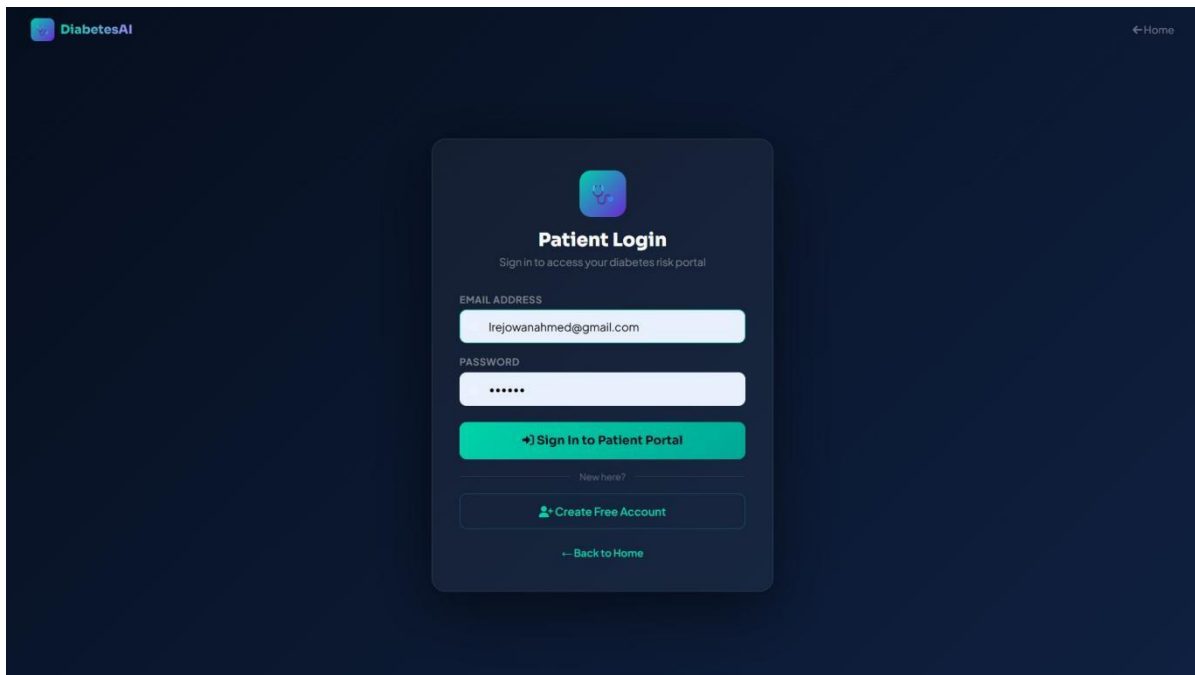


Fig. 3.1.5: Patient Portal (Login and Sign up Page)

Diabetes Risk Assessment

GENDER	AGE (YEARS)	BMI (KG/M ²)
Female	25	27.0
HBA1C LEVEL (%)	BLOOD GLUCOSE (MG/DL)	SMOKING HISTORY
6.6	120	No Info
HYPERTENSION	HEART DISEASE	
No	No	

Analyze Risk with 3 AI Models

Low Diabetes Risk

Best Model: Random Forest
Probability: 0.0% - Model Accuracy: 99.36%

Diabetes Probability

Fig. 3.1.5: Patient Portal (Dashboard)

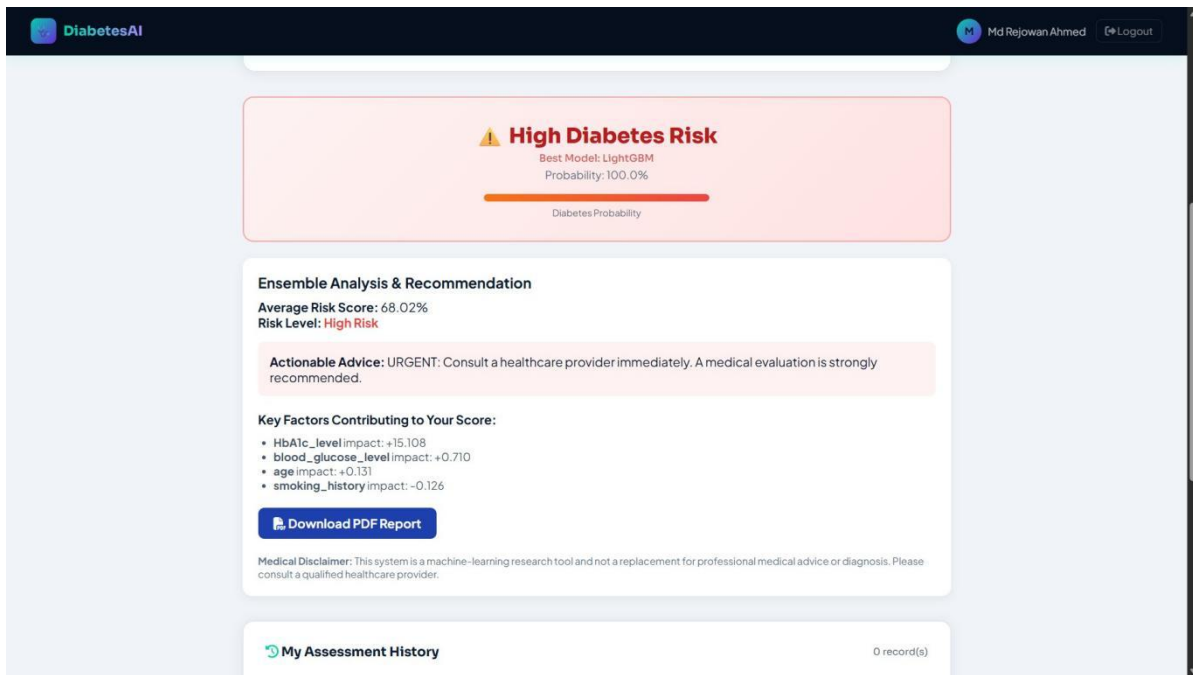


Fig. 3.1.5: Result and Explainable AI

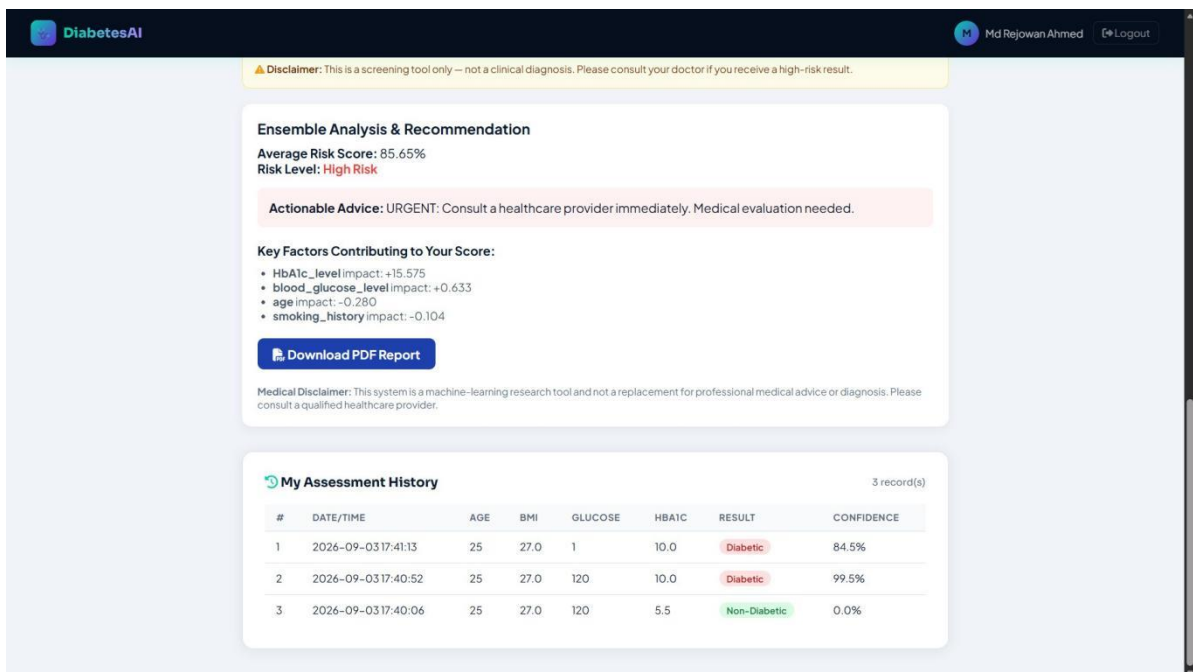


Fig. 3.1.5: Patient Portal (Report PDF file Downloadable)

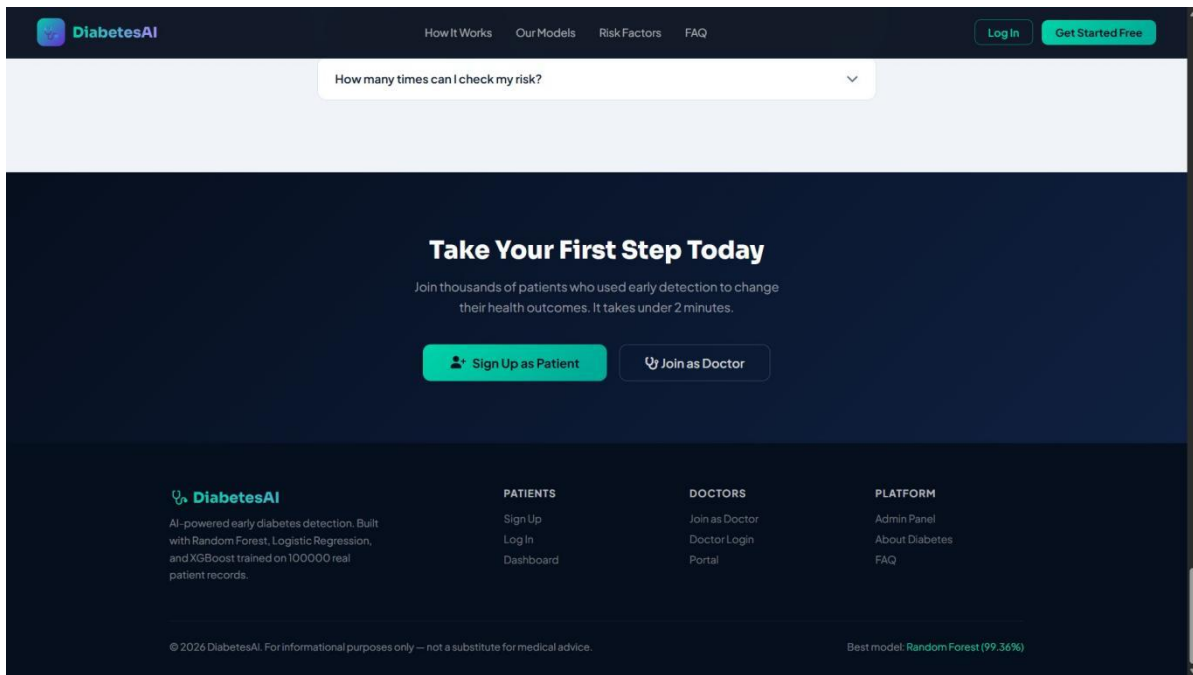


Fig. 3.1.5: Doctor's Section (Home Page)

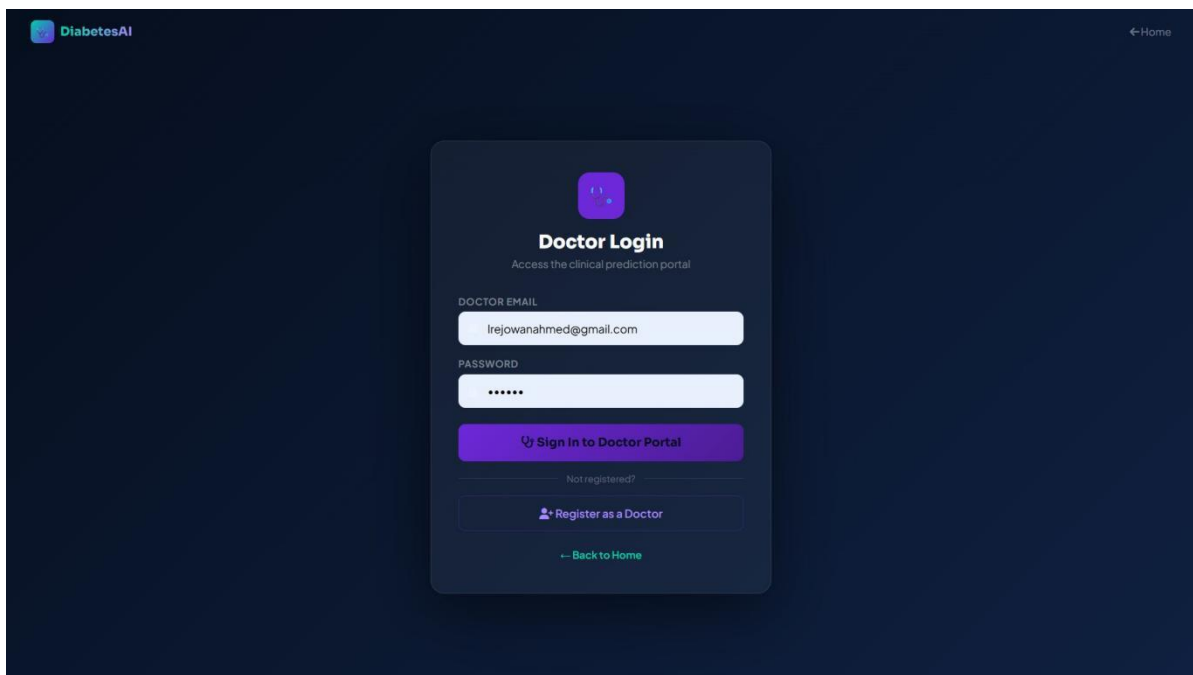


Fig. 3.1.5: Doctor Portal Login Section

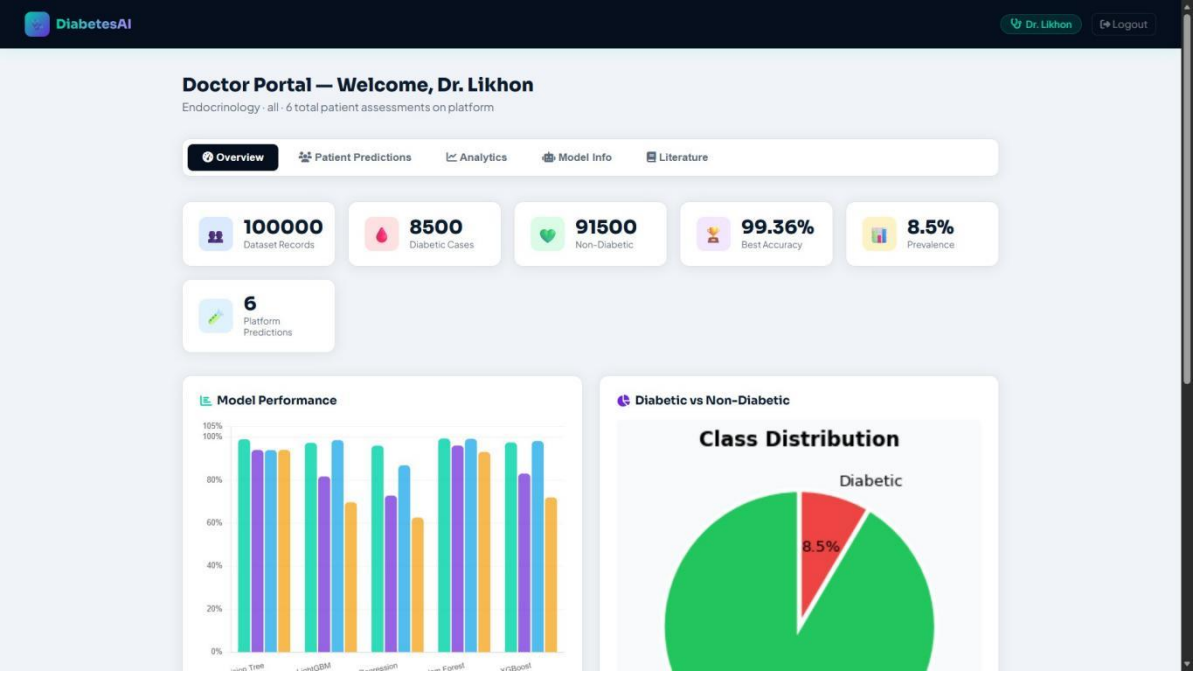


Fig. 3.1.5: Doctor Portal Dashboard

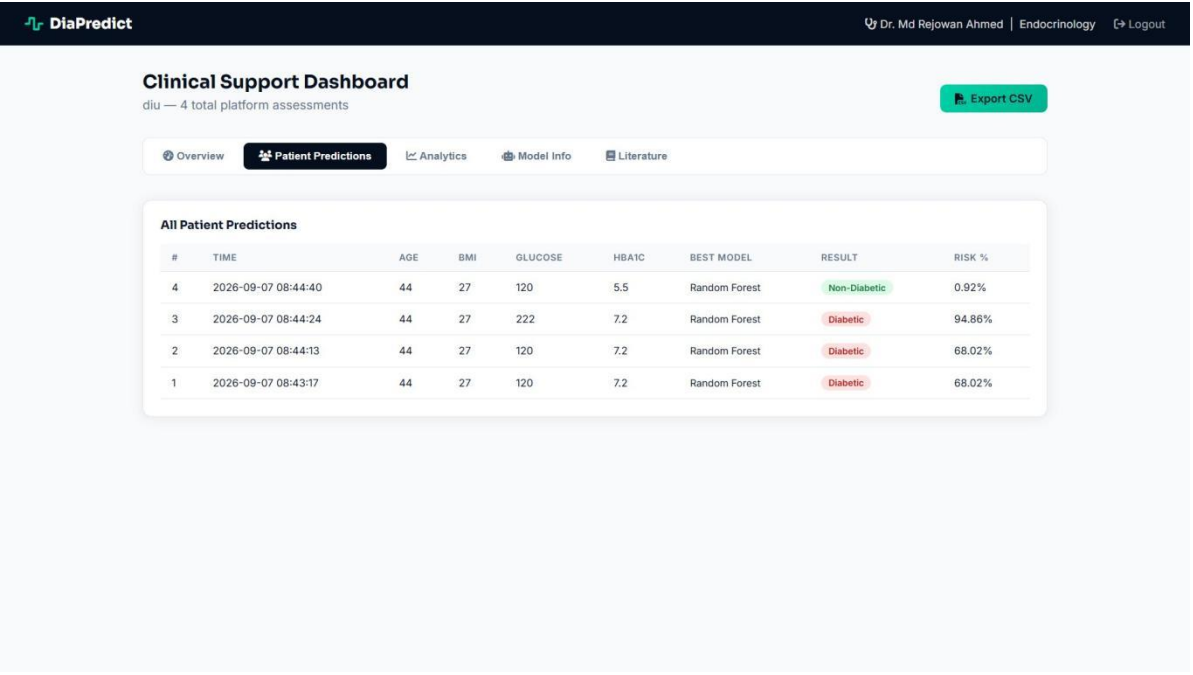


Fig. 3.1.5: Doctor Portal Dashboard (Patient History, Downloadable)

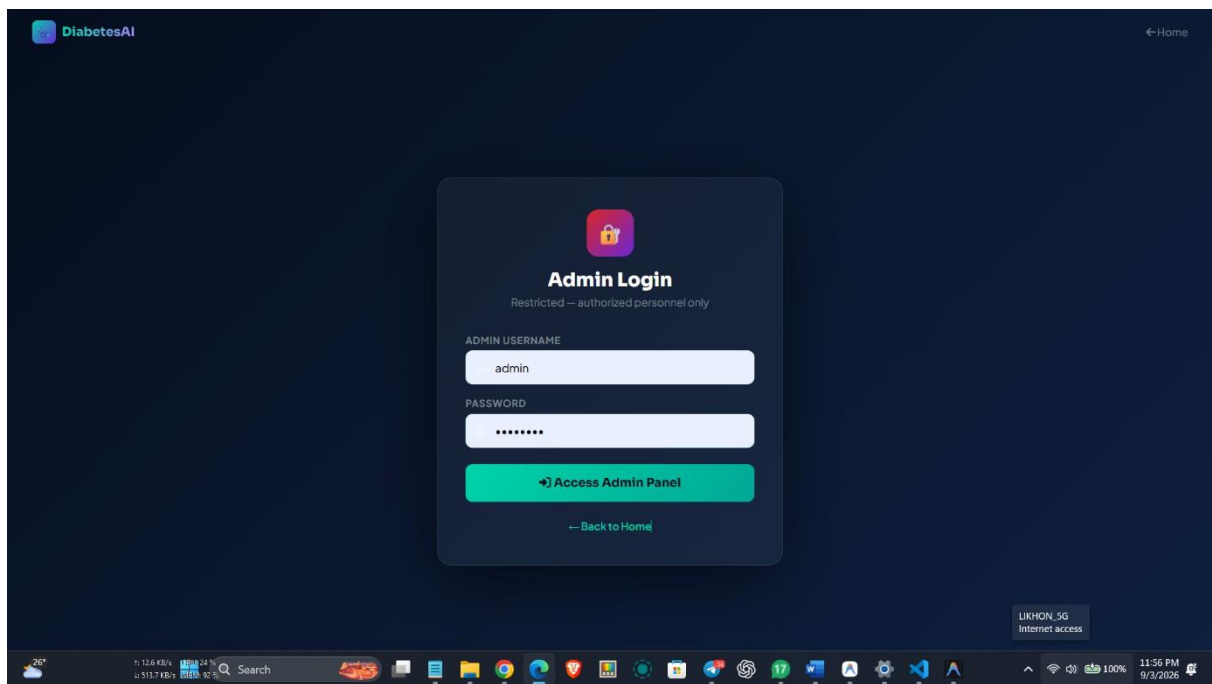


Fig. 3.1.5: Admin Portal Login Section

DiabetesAI

Admin Panel

admin

System Admin

MAIN

Overview

EDA Charts

Model Evaluation

USERS

User Accounts

Prediction Log

Dataset Preview

DOCS

Literature

Public Site

Sign Out

User Accounts

All Systems Active 11:55:09 PM

User Accounts

All registered patients and doctors

Registered Patients

NAME	EMAIL	GENDER	AGE	JOINED	ASSESSMENTS
Md Rejowan Ahmed	irejowanahmed@gmail.com	Female	25	2026-09-03	6

Registered Doctors

NAME	EMAIL	SPECIALTY	HOSPITAL	JOINED
Likhon	irejowanahmed@gmail.com	Endocrinology	all	2026-09-03

Fig. 3.1.5: Admin Portal (user information)

DiabetesAI

Admin Panel

admin

System Admin

MAIN

Overview

EDA Charts

Model Evaluation

USERS

User Accounts

Prediction Log

DOCS

Literature

Public Site

Sign Out

Prediction Log

All Systems Active11:55:17 PM

Prediction Log

Refresh

#	TIME	AGE	BMI	GLUCOSE	HBA1C	BEST MODEL	RESULT	CONFIDENCE
6	2026-09-03 17:49:01	25	27	120	10	Random Forest	Diabetic	99.5%
5	2026-09-03 17:48:55	25	27	120	6.6	Random Forest	Non-Diabetic	0%
4	2026-09-03 17:47:49	25	27	120	5.5	Random Forest	Non-Diabetic	0%
3	2026-09-03 17:41:13	25	27	1	10	Random Forest	Diabetic	84.5%
2	2026-09-03 17:40:52	25	27	120	10	Random Forest	Diabetic	99.5%
1	2026-09-03 17:40:06	25	27	120	5.5	Random Forest	Non-Diabetic	0%

Fig. 3.1.5: Admin Portal User History

DiabetesAI

Admin Panel

admin

System Admin

MAIN

Overview

EDA Charts

Model Evaluation

USERS

User Accounts

Prediction Log

DOCS

Literature

Public Site

Sign Out

Dataset Preview

All Systems Active11:55:22 PM

Dataset Preview

First 15 Records

GENDER	AGE	HYPERTENSION	HEART_DISEASE	SMOKING_HISTORY	BMI	HBA1C_LEVEL	BLOOD_GLUCOSE_LEVEL	DIABETES
Female	80.0	0	1	never	25.19	6.6	140	0
Female	54.0	0	0	No Info	27.32	6.6	80	0
Male	28.0	0	0	never	27.32	5.7	158	0
Female	36.0	0	0	current	23.45	5.0	155	0
Male	76.0	1	1	current	20.14	4.8	155	0
Female	20.0	0	0	never	27.32	6.6	85	0
Female	44.0	0	0	never	19.31	6.5	200	1
Female	79.0	0	0	No Info	23.86	5.7	85	0
Male	42.0	0	0	never	33.64	4.8	145	0
Female	32.0	0	0	never	27.32	5.0	100	0
Female	53.0	0	0	never	27.32	6.1	85	0
Female	54.0	0	0	former	54.7	6.0	100	0
Female	78.0	0	0	former	36.05	5.0	130	0
Female	67.0	0	0	never	25.69	5.8	200	0
Female	76.0	0	0	No Info	27.32	5.0	160	0

Fig. 3.1.5: Admin Portal Dataset



Fig. 3.1.5: Admin Portal Chart Information



Fig. 3.1.5: Admin Portal Chart Information

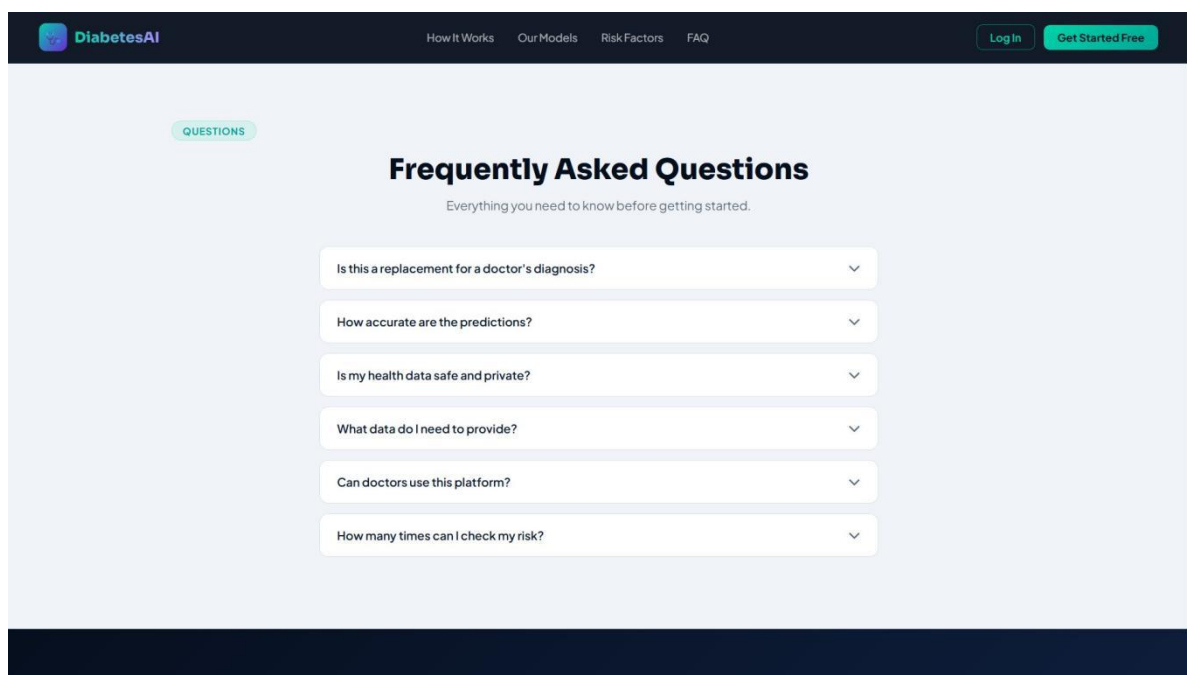


Fig. 3.1.5: Frequently Asked Questions Section (Home Page)

3.2 Detailed Methodology and Design

3.1.1 Dataset Description

The dataset used in this project is the "Diabetes Prediction Dataset" sourced from Kaggle (diabetes_prediction_dataset.csv), a publicly available medical dataset containing 100,000 patient records. Each record contains 8 clinical and demographic features and 1 binary target label:

Table 3.2: Dataset Feature Description

Feature	Type	Description
gender	Categorical	Patient gender (Male / Female / Other) - encoded with LabelEncoder
age	Numerical (continuous)	Patient age in years (range: 0.08 - 80)
hypertension	Binary (0/1)	Whether the patient has hypertension (1 = Yes, 0 = No)
heart_disease	Binary (0/1)	Whether the patient has heart disease (1 = Yes, 0 = No)
smoking_history	Categorical	Smoking status: never, former, current, not current, ever, No Info - LabelEncoded

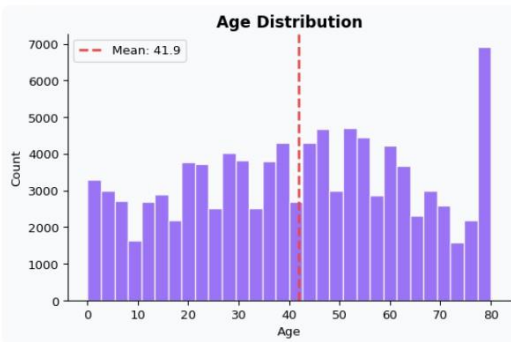
bmi	Numerical (continuous)	Body Mass Index (range: 10.16 - 95.69)
HbA1c_level	Numerical (continuous)	Glycated haemoglobin level - key clinical diabetes marker
blood_glucose_level	Numerical (integer)	Fasting blood glucose in mg/dL
diabetes (TARGET)	Binary (0/1)	Diabetes diagnosis: 1 = Diabetic, 0 = Non-Diabetic (~91.5% class imbalance)

Source: Kaggle Diabetes Prediction Dataset

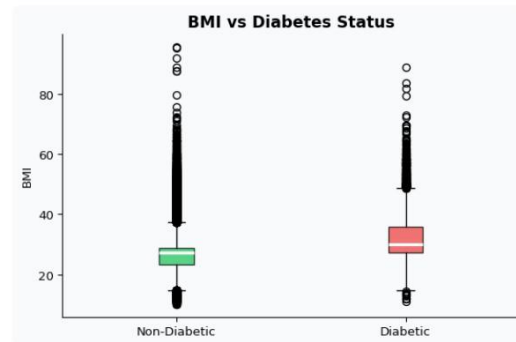
3.1.1 Data Preprocessing

This study is based on the Diabetes Prediction dataset which consists of 100,000 patients' data, each included with 8 input variables which are for example: Gender, Age, Hypertension, Heart Disease, Smoking, BMI, HbA1c, Blood Glucose, and a binary target variable, representing the diabetes status for each patient. Earlier the data set was visually reviewed for any missing values across all columns, and there were no missing values in any of the nine columns. This is because the imputation step (numeric: median or categorical: mode) was put in the pipeline to protect against missing data in future data imports, even if it had no impact on the current data set. The input attributes gender and smoking_history are categorical, and were encoded into a numeric type by replacing each unique category with an integer index. This is preferred v one-hot encoding so that the feature space is not bloated unnecessarily by one-hot encoding features and remain usable for the linear model in the comparison set while preserving very small feature space for the tree-based models. After encoding, all the 8 features have been standardized using StandardScaler which rescaled each of the features to zero mean and unit variance: This step is vital for the coefficient estimates of logistic regression, which are highly susceptible to magnitude changes in the features, and it has a neutral-to-positive effect on the tree-based ensembles. This processed data set was then split into a training set and a test set with 80,000 in the training set and 20,000 in the test set (80/20 split) consistent with another random seed. Interesting to note here, the target classes are imbalanced with 20,000 test set records containing 18,292 non-diabetic and 1,708 diabetic records or 91.5% non-diabetic and 8.5% diabetic respectively. We did not correct the imbalance using any resampling methods in the current pipeline (e.g., SMOTE or class-weighting), which should be discussed directly in the Limitations chapter-this is obvious as it directly explains the low recall on the diabetic class compared to the precision on the five models.

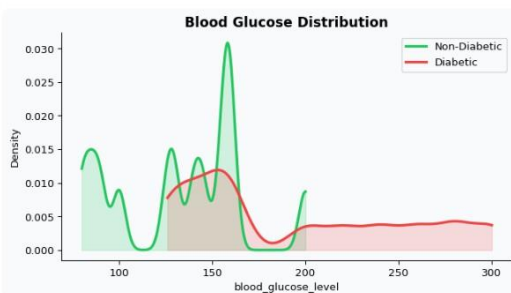
Age Distribution



BMI vs Diabetes



Blood Glucose Distribution



HbA1c Distribution

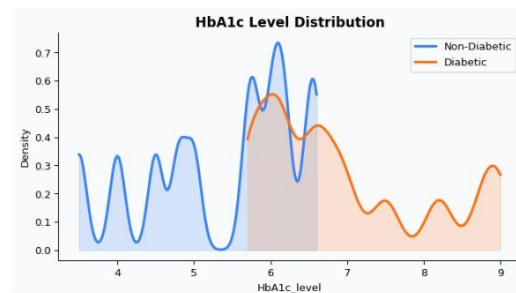


Figure 3.1.1: Dataset Distribution

Feature Importance

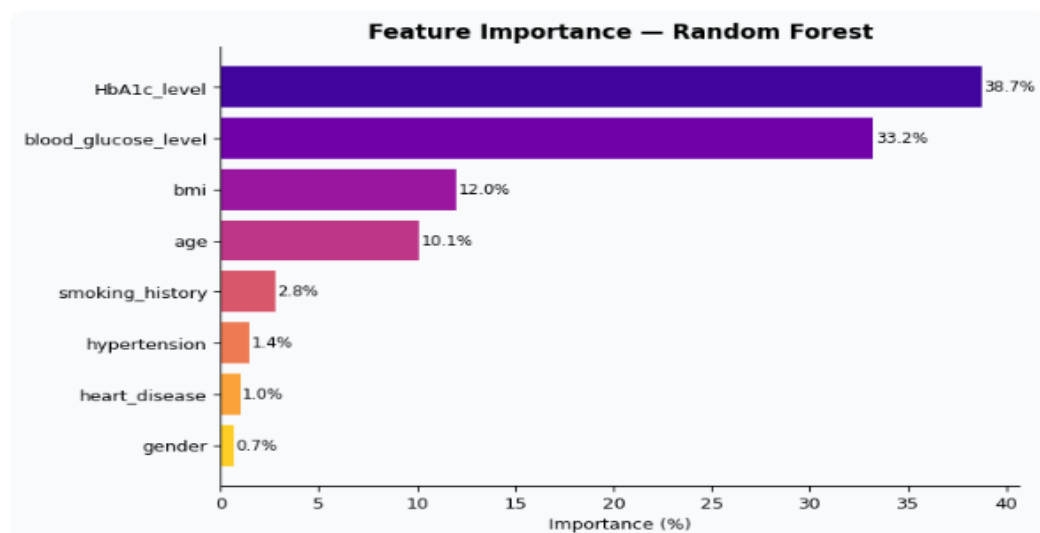


Figure 3.1.1: Dataset Distribution

3.1.2 Model Selection

Given the need to choose the best predictor of diabetes that could classify the diabetes cases, five supervised learning algorithms from different families (linear family, tree-based family and the gradient-boosted family) were trained and compared on the same data splits.

Logistic Regression

Logistic Regression is a linear classification algorithm which estimates the probability that an element belongs to a particular class with a sigmoid function of the sum of feature vectors and weights. Although it is not commonly used as a baseline anymore, the coefficients of it are directly interpretable in the clinical terms of the log-odds contribution of each risk factor, thus facilitating clinical communication of results.

The linear relationship is first calculated as:

$$z = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n$$

The probability of belonging to the positive class is then calculated using the sigmoid function:

$$P(y = 1 | x) = 1 / (1 + e^{-z})$$

The model parameters are typically learned by minimizing the binary cross-entropy loss:

$$L = -(1/N) \sum [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

Decision Tree

A Decision Tree splits the feature space recursively based on a series of threshold-value-based splits that best purify the node according to a usually agreed upon measure, typically Gini impurity or information gain. Easy to interpret but problems of overfitting on unconstrained depth, for which it is used here primarily, as a comparative lower bound against ensemble methods.

For classification, split quality is commonly measured using Gini Impurity:

$$\text{Gini} = 1 - \sum p_i^2$$

where p_i represents the proportion of samples belonging to class i . Alternatively, Information Gain can be calculated using entropy:

$$\text{Entropy}(S) = -\sum p_i \log_2(p_i)$$

The best split is selected based on the maximum reduction in impurity. Decision Trees are highly interpretable but may overfit when tree depth is unrestricted. Therefore, they are used as an interpretable baseline for comparison with ensemble models.

Random Forest

Random Forest is a collection of decision trees, that are built using a bootstrap sample of the original data and a random subset of features used at each split and predictions are

made by majority voting. This ensures that individual trees aren't correlated, which significantly helps avoid the tendency of overfitting one Decision Tree and is able to model interactions between features that create a non-linear relation and to report feature importance scores.

For classification, the final prediction is determined through majority voting:

$$\hat{y} = \text{mode}\{h_1(x), h_2(x), \dots, h_T(x)\}$$

where $h_t(x)$ represents the prediction of the t -th decision tree. Bootstrap sampling and random feature selection reduce correlation between individual trees, helping reduce overfitting and improve generalization.

XGBoost

XGBoost (Extreme Gradient Boosting) grows a tree after each tree trained to minimize the residual loss of the tree it builds, using the L1/L2 penalty to limit the growth of the model. A gradient boosting formulation usually produces a good performance on most clinical data typically represented in a tabular format like the one used in this study.

XGBoost (Extreme Gradient Boosting) builds decision trees sequentially, where each new tree attempts to minimize the errors made by previous trees.

$$\hat{y}_i = \sum_{k=1}^K f_k(x_i)$$

The objective function combines prediction loss with a regularization term:

$$\text{Obj} = \sum_{i=1}^n l(y_i, \hat{y}_i) + \sum_{k=1}^K \Omega(f_k)$$

The regularization term controls model complexity:

$$\Omega(f) = \gamma T + (1/2) \lambda \sum_{j=1}^T w_j^2$$

where T represents the number of leaves and w_j represents leaf weights

LightGBM

LightGBM is a gradient boosting framework with a level-wise growth strategy (as opposed to all leaves) and histogram-based binning of continuous features, and is as accurate or more accurate than the standard implementations of gradient boosting, but it consumes less memory and requires less time for training on large tabular datasets.

The model prediction is formed by combining multiple weak learners:

$$F(x) = \sum_{m=1}^M \eta f_m(x)$$

where $f_m(x)$ represents individual decision trees and η is the learning rate. LightGBM uses histogram-based feature binning to reduce computational complexity:

$$O(N \times \text{Features}) \rightarrow O(B \times \text{Features})$$

where N is the number of data points and B is the number of histogram bins.

Model Performance

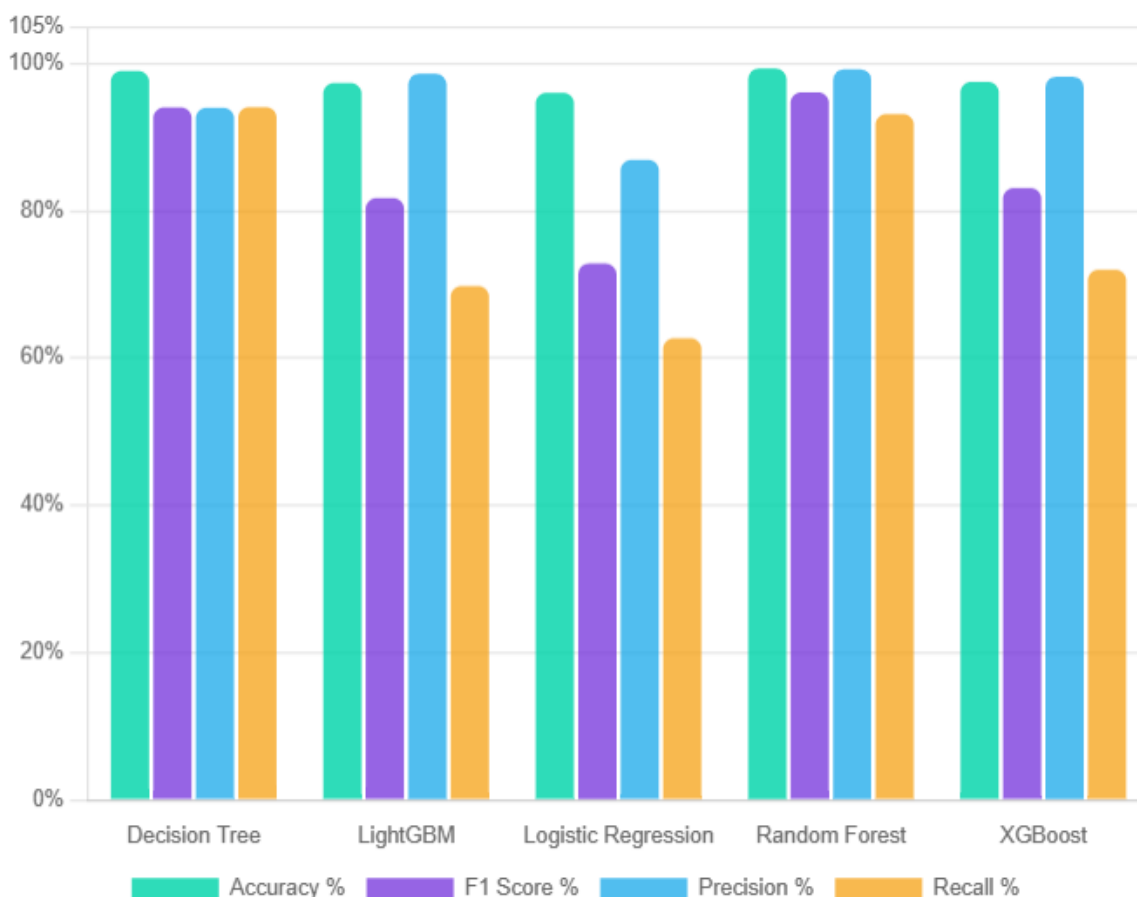


Figure:3.1.2 Model Comparison Graph

3.1.1 Proposed System: Multi-Model Ensemble Diabetes Risk Prediction Framework

This project develops a comparative multi-model diabetes prediction framework using five classifiers trained on the same processed dataset. The best-performing model is identified based on test accuracy, while predictions and probability scores from all five models are presented through a Flask-based web application.

The system uses eight clinical inputs—gender, age, BMI, HbA1c, blood glucose, smoking history, hypertension, and heart disease—and applies the same preprocessing pipeline used during training. It provides model-wise predictions, probability scores, and highlights the best-performing model as the primary recommendation.

To improve interpretability, SHAP explanations show how individual clinical features influence predictions, along with global feature importance for LightGBM. The system also includes Patient, Doctor, and Administrator dashboards, with patient reports, prediction history, model performance metrics, confusion matrices, ROC and Precision-Recall curves, and administrative data management.



Figure 3.1.1: Confusion Matrix and SHAP summary

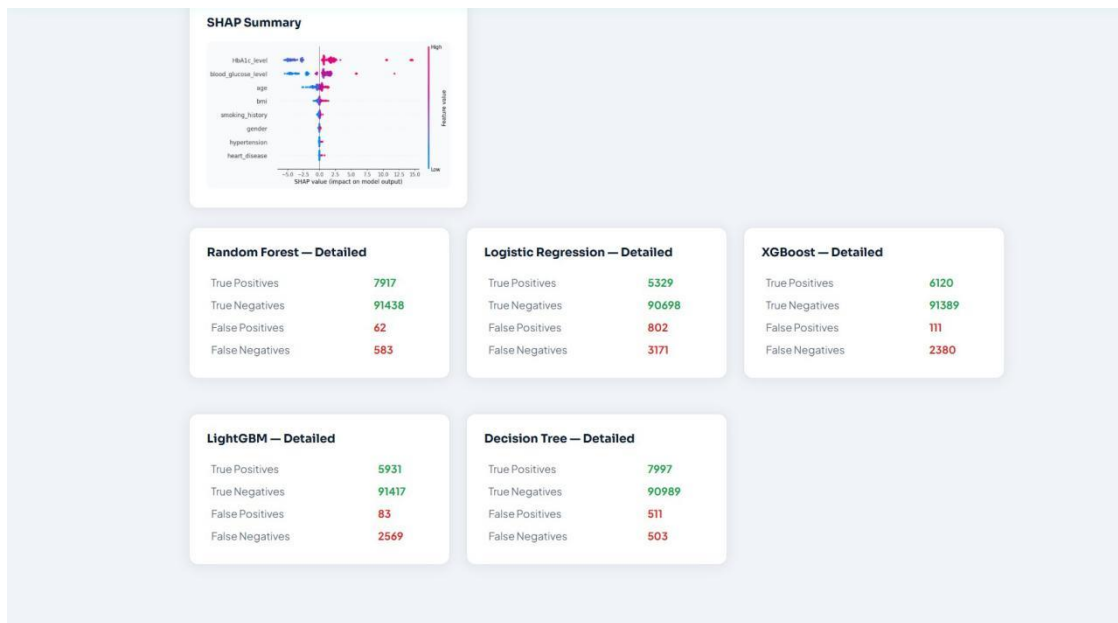


Figure 3.1.3: Multi-Model Ensemble Results

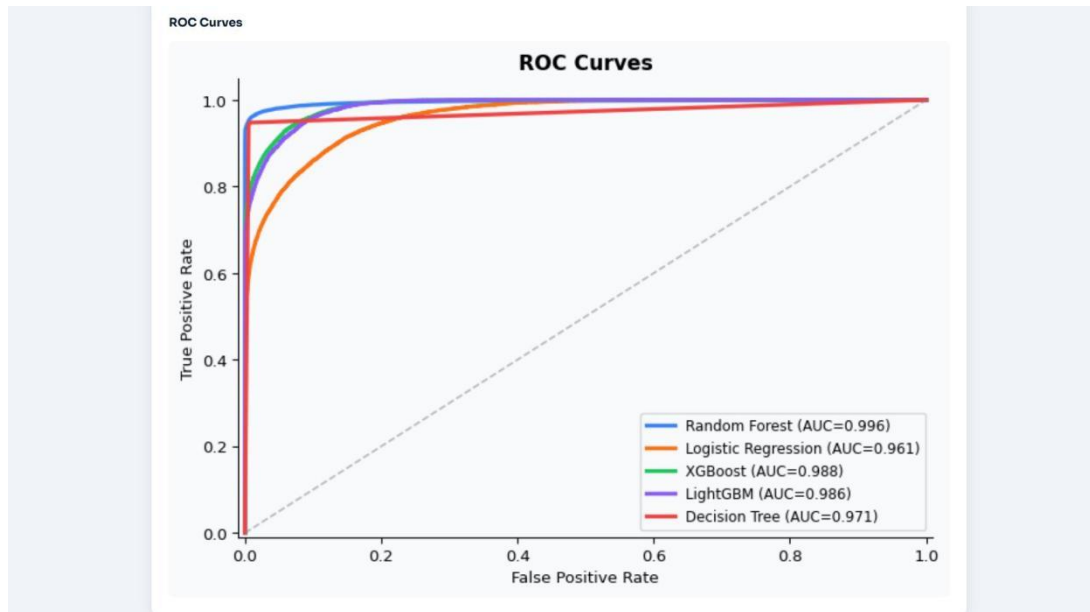


Figure:3.1.5 ROC curve

3.3 Project Plan

- Data Collection & Understanding: Analyzed dataset structure, distributions, missing values, class imbalance, and data quality.
- Data Preprocessing: Cleaned data, encoded categorical variables, and prepared features for training.
- Model Training & Testing: Trained and tested five ML classifiers under the same conditions for fair comparison.
- Web Application Development: Built the Flask application and integrated ML models, predictions, SHAP explanations, and role-based interfaces.
- Integration & Testing: Integrated all components, tested features, and evaluated system performance.
- Deployment & Documentation: Deployed using Ngrok and documented the development process and results.
- Teamwork: Divided responsibilities between two members with joint integration and testing.

3.4 Task Allocation

To ensure a workload balance, and to be able to develop the project work in parallel, the project work was split between the two members of the team. The data set analysis, pre-processing, feature engineering, building machine learning models, performance testing, and explaining the models with the SHAP approach was done primarily by one member. The other members worked primarily on the development of Flask applications, user-role management, their front end design, dashboard development and deployment of the system. Integration, debugging, testing, performance verification, research

documentation, and preparing the final report. Various issues of model choice, system architectures, implementation of features, and end-testing were jointly discussed and reviewed to ensure consistency and reliability throughout the project.

Table 3.3: Task Allocation

Tasks	Weeks																		
	12	14	16	18	20	22	24	26	28	30	32	34	36	38	40	42	44	46	48
Dataset Acquisition & EDA																			
Model Training																			
Evaluation, Flask Web App Development																			
Admin Dashboard																			
RBAC Implementation																			
Ngrok Deployment & Testing																			
Final Report Writing & Review																			

This table depicts the timeline of the principal activities in each period of the project from 12 to 48 weeks.

3.5 Summary

In this chapter we have discussed the complete methodology behind the project, beginning with requirement analysis of the system which had identified the patient, doctor and admin needs separately. This was followed by system design which involved not using a single classifier but designing the system based on five models being run simultaneously. We have also discussed the functional and non-functional requirements of the project and the interface design of the system as well. The other methods which were considered but were ultimately discarded were those of single model prediction, rule based scoring system, immediate database implementation and enhanced password hashing technique and reasons for choosing certain methods over others were also discussed. All of the trade-offs made in order to get a viable system were based on the premise of building the system in the time allotted for Phase I and making note of areas that needed further improvement later on.

Chapter 4

Implementation and Results

The aim of this chapter is not just on what it was designed to do, but really on how the trained models and the overall application they will be integrated in actually perform when set up. It opens with the setting that the work was undertaken in, progresses to a discussion of the objective measures and comparing theirs that was used to assess each of the five models, then explains the meaning of all those results (including a couple of inconsistencies that it is best to keep in mind since it talks up); and terminates with a brief summary.

4.1 Environment Setup

All the development, training and initial deployment for this project were done on Google Colab, benefiting from its free-tier runtime with attached Google Drive where datasets were stored and the model was persisted. The processing pipeline of the machine learning part of the application has been created using python 3, scikit-learn, XGBoost, LightGBM libraries and ngrok has been used to tunnel the locally running Flask server to a public URL for testing on a browser.

Table 4.1: Development and Deployment Environment used throughout the study.

Resource Category	Specific Resource	Purpose
Computing Platform	Google Colab (free tier, Python 3.x runtime)	Development, training, and deployment environment
Dataset	diabetes_prediction_dataset.csv (Kaggle)-100,000 records,8 features	Primary training and evaluation dataset
Python Libraries	pandas, numpy, matplotlib, seaborn, scipy	Data loading, manipulation, and EDA visualization
ML Libraries	scikit-learn (RandomForest, LogisticRegression, metrics), XGBoost	Model training, evaluation, and scoring

Web Framework	Flask (Python)	RESTful web application backend development
Deployment Tool	pyngrok / Ngrok	Public URL tunneling from Google Colab
Frontend Libraries	Chart.js, Font Awesome, HTML5/CSS3/JavaScript	Admin dashboard charts and UI components
Model Persistence	joblib	Saving and loading trained models and preprocessors
Version Control	GitHub	Dataset storage and model artifact backup

The data in a pipe did not need much data cleaning at the record level: There were zero missing values found for all nine columns, and median/mode imputation logic is implemented as a precautionary measure in the pipeline, rather than changing the data in this case. Firstly gender and smoking_history were one-hot encoded using LabelEncoder and subsequently all the 8 input features were normalized using StandardScaler before being input to the models. The data set was randomly divided into training and test sets (test_size = 0.2, random_state = 42), 80,000 cases were used as training, while 20,000 cases were withheld for testing, including 1708 diabetic patients and 18,292 non-diabetic patients.

4.2 Testing and Evaluation

The 20,000-record held-out test set was used in each of the 5 models. The metrics examined were accuracy, precision, recall and f1 score along with a confusion matrix for each model. These four metrics were selected intentionally, as the dataset is significantly class imbalanced with only 8.5% of the examples being for a "diabetic" case. This means that using accuracy alone as a measure of the model may involve it making an incorrect prediction just because it predicts "non-diabetic" the majority of the time. The precision and the recall of the model (especially that of the diabetic (positive) class) more closely reflect the true utility of the model in the clinical domain.

The following preliminary results were obtained from model training and evaluation on the test set (20% of 100,000 records = 20,000 test samples):

Table 4.2: Model Performance on Test Set

Model	Precision	Recall	F1-Score	Accuracy (%)
RandomForest	0.95	0.69	0.7996	97.05%
Logistic Regression	0.86	0.61	0.7171	95.87%
XGBoost	0.95	0.70	0.8055	97.13%
LightGBM	0.98	0.69	0.8104	97.24%
Decision Tree	0.71	0.74	0.7243	95.21%

4.2.1 Precision

The number of the samples, which were correctly identified as a certain type of disease, was calculated with the help of Precision (as provided in Equation 1). It has been measured as the true positives, divided by the true positives as well as the false positives. The higher the precision score, the higher the chances that when a model forecasted a positive result it was rather right and did not generate too many false alarms.

$$Precision = \frac{TP}{TP+FP} \quad (1)$$

4.2.2 Recall

The Recall (Equation 2) was used to decide which number of actual disease-positive cases were completely detected by the model. Given that this is more medical focused study, the higher the value of the recall, the better. It makes sure that cases with diseases were not neglected.

$$Recall = \frac{TP}{TP+FN} \quad (2)$$

4.2.3 F1-Score

F1-Score (Equation 3) can be considered to be used as an amalgamation of both precision and recall. It was applied particularly because in some instances a trade-off might occur between the quantity of true positive results (recall) and the accuracy of such forecasting (precision). The reason why the harmonic mean of these metrics was also computed was to present the whole picture.

$$F1\ Score = 2 * \frac{Precision*Recall}{Precision+Recall} \quad (3)$$

4.2.4 Accuracy

Accuracy (Equation 4) was used to determine the number of the all predictions made by the model that were accurate. It took into account correct negatives and correct positives. However, it was restricted by other metrics since inaccurate accuracy of some cases may occur due to the imbalance of the classes on the medical data.

$$Accuracy = \frac{TP+TN}{TP+TN+FP+FN} \quad (4)$$

4.3 Results and Discussion

Combining all of the results, the outcome is fairly clearly LightGBM is a very good all-rounder on this data, followed closely by XGBoost and the Random Forest, and behind by a significant margin from Logistic Regression and the Decision Tree. What's important to discuss is not only which one won, but why the difference between the top 3 and the bottom two exists and what it indicates that it means for the system that is supposed to be used by actual patients.

Mainly the difference is down to how both algorithms tackle the class imbalance and how they are applied to the non-linear relationships of the data. Logistic Regression has a linear form, so it is restricted to drawing a linear boundary between the features and is trained on less than 10% of the cases, leaving it conservative in its decision: it fails more in recalling a diabetic case (0.61) than any other model here, which is the worst possible failure mode for a screening tool, since a missed diabetic case is a case exactly the system is supposed to avoid. On the other hand, the Decisive Tree fits too tightly to the training data and fails to generalize to samples which end up with a higher error on the false positive side, leading to a lower precision (0.71) of the tree in the test sample, despite having the best recall (0.74) of all the five trees.

Correlation Heatmap

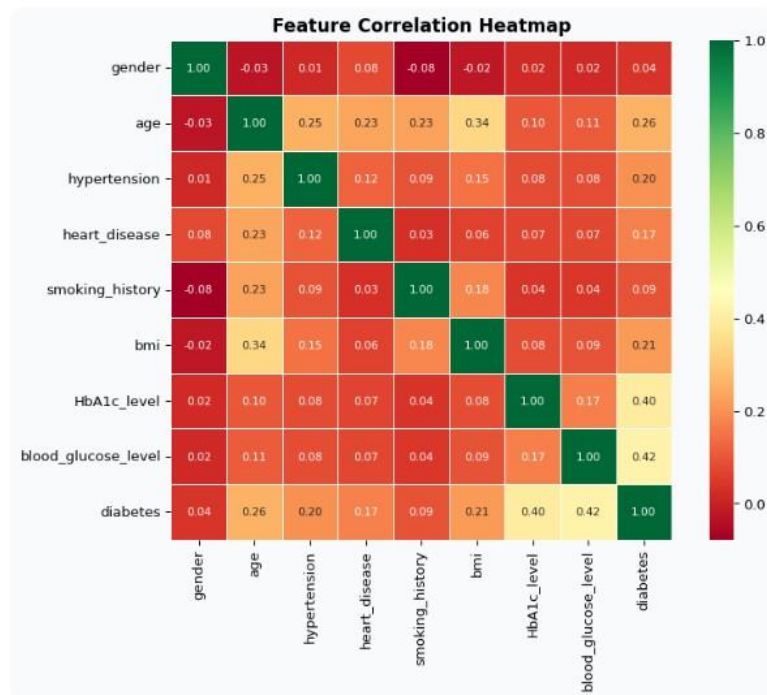


Figure 4.3: Correlation Heatmap

The three ensemble methods - Random Forest, XGBoost and LightGBM - fall somewhere between these two types of failures, in terms of handling the trade-off. LightGBM's slight improvement over the other two is consistent with other findings that gradient-based approaches performed better than simpler baseline methods when trained on somewhat similar diabetes datasets with similar number of instances.



Figure 4.3: Confusion Matrix

Aside from the performance already noted, a SHAP summary of the LightGBM model also confirms that these are the two top features that influence each person's actual

prediction most: HbA1c level and blood glucose level are the most important features, while BMI and age are also significant, and gender and smoking history are far less significant individually. This is consistent with clinical intuition (fasting glucose and HbA1c are the most commonly used tools for diagnosing diabetes in practice), and as Sun et al. found in the comprehensive study they conducted, which also saw HbA1c being recognised as the single most significant variable in all the models they examined. This convergence on the same two features between an independently trained model produced using a different dataset is a bit of confirmation that the system is seeing real clinical signal instead of some artefact unique to this dataset.

On the deployment side, the system did what it was supposed to do: When a patient type a number on one of his clinical values on the assessment form, the system predicts five numbers, an ensemble-averaged risk score, a recommendation on the risk level and a SHAP explanation of his own prediction (all in a few seconds), and lets the user download a pdf of the result. The doctor and admin dashboards appeared populated with the exploratory charts-with correct model metrics to compare-in addition to a live log of predictions made via the patient portal during the test. Both were reported in some of the challenges during Phase-I testing, and both are planned to be tackled when making the gradual transition towards a more stable deployment, but are not modeling problems; on the deployment side - Ngrok's free side sometimes needed to be restarted during testing - and the port used by Flask (5050) was found to be already in use in at least one test run.

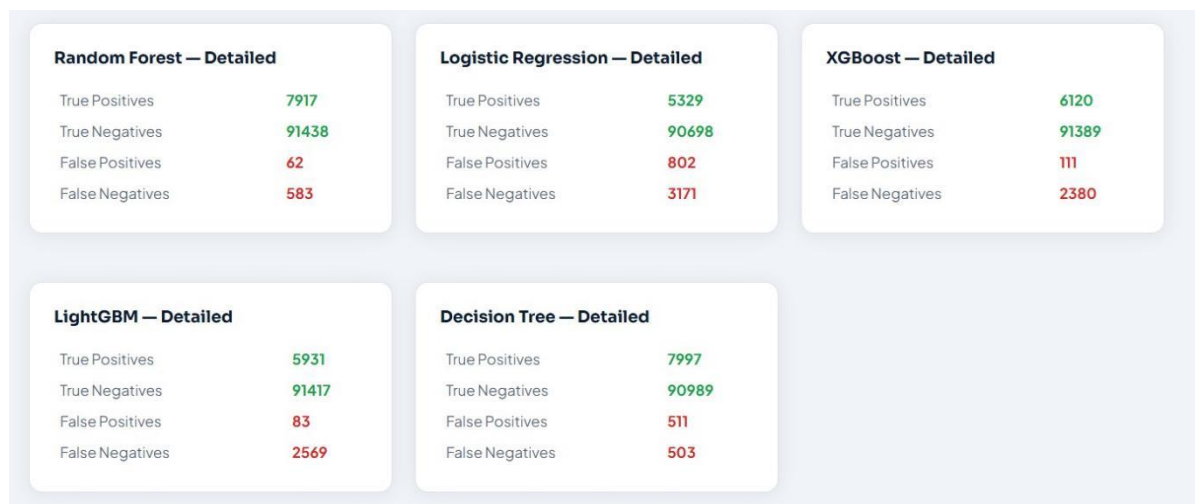


Figure 4.3: Five model evaluation

4.4 Summary

This chapter demonstrated that five machine learning models - Random Forest, Logistic Regression, XGBoost, LightGBM, and Decision Tree - when trained on the 100,000 record diabetes dataset and compared, the XGBoost model performed best, and that this model was wired into a deployed, multi-role web application. In the other test set which was not used in the training process, LightGBM achieved the best overall balance between Precision and Recall (81.04% F1-score and 97.24% accuracy), slightly surpassed XGBoost and Random Forest, whereas LR and DT were not quite faring better in opposite ways - one being more conservative in the risk signal it was issuing, while the other was less. But there is a real held out test accuracy that doesn't match the "live" accuracy that appears on the admin dashboard (which would be updated with full-live data, but not just test data). In addition to number counts, the explainability from SHAP aligned well with clinical and previous published results in identifying HbA1c level and blood glucose level as the two primary influences on the predictive model, while the application itself (patient portal, doctor dashboard and admin dashboard) worked as expected on testing, with the key pain points not being modeling-related but related to deployment. The combined results leave no doubt that the system should rely heavily on, and should be amended to rely on more heavily, LightGBM in the future and provide details that show precisely where LightGBM is being used in the dashboard and in this study's current implementation needs to be corrected before the system can be taken to the next stage beyond a Phase-I demonstration.

Chapter 5

Engineering Standards and Design Challenges

This chapter examines the project on an engineering-standards and a more general impact perspective. It covers the software and hardware standards and methods used in the design of the system, discusses the communication standards and methods used, explores the societal and ethical implications of the project, discusses the budget/expected revenues, and maps the project to the broader engineering problem and engineering activity frameworks used for assignments in the final year of Design.

5.1 Compliance with the Standards

The development of any health-related Web application, even for purposes of an undergraduate final year project, has an ethical responsibility to use recognised engineering practice and avoid the 'cattle and water' approach. Each decision on software libraries, deployment hardware and communication protocols was not a random one and was considered against at least one alternative, most of which are presented here with the standards against which they were made.

5.1.1 Software Standards

The software aspects of this project was guided by ISO/IEC 25010:2011 (international standard for software product quality) on what “good” software should measure; functional correctness, performance efficiency, usability, reliability, and others. security, and maintainability. This should not be taken as a to-do list, rather it gave rise to some concrete decisions, such as ensuring that elements of the preprocessing pipeline (FEAT_COLS, the LabelEncoders, the StandardScaler) are kept the same across training and inference, as drifting away across these elements without detecting error will be a problem of reliability. We have also considered writing all the Python code in accordance with PEP 8 conventions, partly to preserve our sanity while collaborating with two people, with part of it as well being the de facto common standard for Portuguese readable and maintainable Python.

On the framework side, I had to pick either Flask or Django or FastAPI before any code was written and none of the three are magic bullets. On the framework side, I had to make a realist choice amongst the 3 frameworks (Flask, Django, FastAPI) before I started writing any code, and none of these are magic bullets. Things are enhanced by a full ORM, built-in admin system and a fairly rigid project structure, which is all beneficial

for a large production application, but not much for a two man team looking to get some initial prototypes up and running inside a single google colab notebook. Consideration was given to the use of FastAPI as well, primarily because of its built-in support for asynchronous operations and automatic API documentation, but because they were less used to its asynchronous operation paradigm the team was more comfortable with the synchronous API documentation model, provided by Flask. A major reason why Flask was selected was that everything-routes, HTML templates (which are used through `render_template_string`) and the actual prediction logic-can be executed in a single Colab notebook using the 'training and serving' paradigm employed here. A comparable scenario repeated itself, this time on the modeling end with classical machine learning on the one hand, and deep learning on the other hand. Since the entire data set consists of table data with no image or sequences, first the tuning and learning of a deep neural network would do much better than tree-based ensembles, and, second, because the interpretation of these tumours would be far less transparent than the tree-based ensembles, a property very desirable for a health application where a patient or doctor would be happy to ask "why did it say I'm at risk?". The libraries used instead for the two boosting models are XGBoost and LightGBM; for the three other models, (Random Forest, Logistic Regression, and Decision Tree), classical ML libraries were selected, partly because they were what was consistently reported in the literature covered in Chapter 2 as the models that provide the most reliable performance when handling structured medical data, and partly because tree ensembles naturally lend themselves to SHAP for explainability, from the start of the design, which is a requirement.

Hardware Standards

Since the system operates on the web, rather than on a dedicated hardware entity that gathers its own sensor data, it is considered that the hardware standards that are relevant here are those of the computing environment (web server, etc.) in which the system operates, and those of the network hardware upon which it relies for access to the system.

As for the training component, the project used Google Colab's provisioned hardware which consisted of a 16 Gb GDDR6 GPU from NVIDIA with CUDA Compute Capability 7.5 and approximately 12 to 16 GB of system RAM. It is important to note that of the five models evaluated here, none is deep neural networks, and that, during experimentation, the GPU was used to speed up experimentation, rather than the requirement, as all five models could run inference on standard CPU hardware without any problems in production, which would keep the deployment cost low. An on-premise dedicated GPU server was briefly mentioned as an option but abandoned very quickly: it would have been a real physical cost and a physical burden for a two person academic team, working a large percentage of the time remotely, that could not reasonably manage, for a benefit (increased training speed of models that are already extremely quick to train), that would not really justify the expense.

From the client side, there are no special end user hardware requirements specified. All the interface is HTML/CSS/JavaScript rendered via Flask and all that is required from a

client is a device and modern web browser.

The system relies on standard IEEE 802.11 (Wi-Fi) and IEEE 802.3 (Ethernet) to connect to the public internet since the deployed application is only accessible via the Ngrok tunnel when it is running. There was no special network equipment that was necessary beyond what any home or office internet connection offers - intentional as this project is being ultimately targeted to the clinic environments that may have a lower resource base.

5.1.2 Communication Standards

This project uses virtually all kinds of regular Web protocols for communication, because this project consists of a Flask-based backend providing REST-like endpoints accessed by JavaScript in the patient's browser. All information passed across from the frontend to backend is conveyed in the JSON (RFC 8259) format, de facto standard for such an API request/response over the past few years or so.

The application is exposed via an externally accessible URL, Ngrok (port 5050) is a plain-HTTP server but the traffic between a user's browser and the public URL is carried out on an encrypted channel. It is worth stating as bare facts and not simply passing over, it is a fine balance to be acceptable provided TLS termination takes place anywhere but at the tunnel: it is one of the security aspects called out in the future work section of the discussion that follows in this chapter.

There were two alternative methods of communication, which were rejected. WebSockets was considered briefly, since it has full-duplex communication that is real time, but the low-level nature of this interaction-patterns (patient fills out a form, waits a few seconds, provided with a result) is very simple request-response and does not require a persistent open connection; the addition of complexity of WebSockets for this simple system did not provide the advantages; the advantages were shown to be negligible after a short while. REST was also mentioned as GraphQL is more flexible with queries, but its disadvantage is that the endpoints are quite strict and limited in number, whereas REST was better suited for a two-person team working under a strict time constraint, with well-defined endpoints like prediction, login, dashboard data, prediction log. So FTP wasn't really a contender - there's no other requirement for mass file transfer in this system that wouldn't be fulfilled by the occasional PDF report download, which is already transferred via HTTPS as all else, other functionality aside.

5.2 Impact on Society, Environment and Sustainability

This section invites the examination of the system not in terms of the technology questions raised, but whether the system serves a purpose and why it was built, going beyond a course requirement. It is a very simple diabetes screening programme and the approach in which it was constructed - the low cost and use of open tools, and an open design that focuses on explainability from design from the outset - was an intentional effort to keep the public-health need accessible to implement, rather than a purely academic exercise.

5.2.1 Impact on Life

More than 537 million adults are diagnosed with diabetes now and the prevalence of diabetes in Bangladesh has been steadily increasing due to changing diets and lifestyle due to urbanization and growing population, access to specialist diagnostic services is uneven and many people don't achieve good glycemic control. Clarification here of what this system does and does not do is important: it's not a replacement for the blood tests which cost time to conduct, namely to measure HbA1c and glucose, but rather these two values are necessary inputs to the prediction. It provides, instead, a methodology to interpret those values, when they have already been obtained from a patient, according to 5 models using a variety of approaches (training methods) that are also trained independently, not in a fixed manner, and explains what are the factors causing the obtained result. That is a key step, because for someone with a regular blood test but hasn't yet seen a specialist, it's a step from a raw lab test to an indicator that can actually help them take action either before or at the point of an appointment; it also provides them with something to make it easier to use during an appointment, whether it's a downloadable PDF or an ensemble risk category or a simple recommendation in regular language. For a doctor, the utility remains in a second layer: The doctor can view prediction patterns simultaneously in many patients at once in the doctor's own dashboard, versus one patient at a time.

5.2.2 Impact on Society & Environment

The overall aim of the project is to reduce disparities in diagnostic assistance services in between well-equipped hospitals and smaller clinics or rural health centers. The combination of the entire stack being OSS, the fact that all training and initial deployment was done using free infrastructure (Google Colab and Ngrok) puts on paper a possibility for any affordable clinic with very little budget to run a version of this system without resorting to the use of costly diagnostic software. Even being able to access the current version of the project is still a prototype, just Phase-I, but it is the biggest societal change that the project is working towards, if it wasn't something that would be handed to a hospital tomorrow.

From an environmental perspective, this approach with classical tree-based ensembles, rather than deep neural networks, represents a real albeit minor sustainability gain. Unlike large neural networks that sometimes need dedicated GPU resources to train on and serve from, the said models can be trained in seconds to minutes and can perform inference on typical CPU devices on this dataset. When multiplied by the number of clinics and the number of predictions per day, that time differential is far from minor and that, as well as being a factor in interpretability, formed part of the reason why they didn't go deep-learning on a problem that didn't need it.

5.2.3 Ethical Aspects

None of the actual patient records were used to train the models in this project because the project used a publicly available, de-identified Kaggle dataset. But this is not the same with the web application deployed - as real patients and doctors sign up, real patient/person information (names/person, emails/person, ages/person, phone numbers/person) does enter the system, and it's not something to be overlooked. However, at present it's only stored in memory for the duration of the application running - something that could be exploited in a brute force attack if multiple instances were running - and passwords are hashed using SHA-256 without salting which while this is a necessary security step there are other password hashing routines that are more complex and are considered more secure (such as bcrypt and Argon2) and these are also explicitly stated as something to be done before any real deployment but not something to claim is already covered.

Another consideration that needs to be honest is fairness. This training set consisted of many patients but was not specifically representative of the demographic and clinical characteristics of the Bangladeshi population and this project did not validate or recalibrate the models with local data. Finalising such a statement for a prediction system applied to any and all populations it can be applied to is incorrect and the point of this limitation will be made rather than left as an assumption in the future work discussion as you may find in other chapters. There is, however, a more positive side to the introduction of SHAP in that the implementation of the SHAP itself was an ethical decision, not just a technical one, and that is a system that informs a patient that she is high risk - without it explaining what it was that made it that way is much less trustworthy and contestable than one that tells her, feature by feature, what caused this diagnosis.

5.2.4 Sustainability Plan

- Technical Sustainability: Flask application is modular coded - each model stored independently using joblib and have the ability to add new models and/or clinical features without the need to recode the prediction pipeline from the ground up. This is important because diabetes-prediction research continues, and any system that can't get a better performing model to work a year from now won't be very useful for much longer.
- The one user reported direct cost of this project was internet connectivity research costing an estimated BDT ~2,900 for the phase-I of this project (which amounts to 58% of the total cost). The other cost was related to the Google Colab Pro access and it came out to be the cost of printing/Documentation and it is BDT ~4,202 for Phase-I of this project. The cost of printing/Documentation for Phase-II was not estimated because it is expected to be the same as phase-I since the amount of produced documents will be the same. This is meant to be a low barrier to entry because this type of system, this type of software is one that a small clinic could see itself step by step in the environment that a small clinic could see itself working, where a diagnostic, one that required an enterprise license, was not.
- Environmental Sustainability: Selecting a lightweight tree-based model, rather than a deep learning approach, also keeps training and inference computationally cheap - and truly sustainable given the simplicity of the dataset, which didn't call for the added representational power of a neural network.

- Community and Knowledge Sustainability: SHAP-based explanations embedded in the portal act as a mini educational tool, allowing a non-technical end user to see, in simple terms, which of their own health measures (such as HbA1c, glucose, BMI, etc) are most important. In smaller ways too, the literature review and gap analysis conducted in Chapter 2 contribute to the already extant academic conversation: what a truly deployable diabetes-screening system ought to be like goes way beyond this single codebase.
- The project needs to be managed and analyzed for financial aspects

5.3 Project Management and Financial Analysis

All the costs associated with this project went into the Phase-I design, where all additional costs were kept deliberately low, relying on low-cost open-source software/vs and free open-source cloud services used wherever possible. The estimated cost breakdown of Phase-I during the development process is as of the table below.

S.No	Resource Item	Est. Cost (BDT)
1	Google Colab Pro (if required for extended runtime)	~1,200
3	Stationery and Documentation	~200
4	Internet Connectivity (both members)	~1,000
5	Printing & Submission Costs (report photocopies)	~500
Total Estimated Cost for Phase-I		BDT ~2,900

There was a general discussion of experimental ways in which a system such as this could sustain itself economically after being outside of a laboratory-based academic project, but no such solutions have been realized here or tested. This would allow the patient-facing risk check to remain FREE, with a subscription fee for clinics or individual doctors to access the doctor dashboard, full prediction history, or for generating a PDF of the prediction history at scale. A different approach would be a B to B SaaS methodology whereby clinics or diagnostic centres would pay a monthly subscription fee for access to the whole platform - patient portal, doctor dashboard and admin oversight - as an additional screening tool to the clinics' existing service, but not as a substitute for any. Another less commercial suggestion is to collaborate with a NGO or the government health department for the deployment of the tool at public screening camps; this is the original motivation behind the project to enhance access in places that were less

commercialized.

5.4 Complex Engineering Problem

The project addresses a complex engineering problem by integrating machine learning, explainable AI, and web application technologies into a unified diabetes prediction framework. The system should be capable of handling a large imbalanced dataset from healthcare, compare multiple classification algorithms, provide reliable prediction performance and present the results using an accessible role-based web interface. It also entails addressing practical aspects of model interpretability, data handling, security, system integration and deployment, making the project a multidisciplinary engineering solution, rather than a standalone machine learning model.

5.4.1 Complex Problem Solving

This is no easy task to bring together the 5 independently-trained classification models, along with the 3-level authentication system, an explainability layer and a live deployment pipeline, and the table below illustrates how this project compares with attributes common to Engineering Problem assignments associated with final-year design work.

Table 5.1: Mapping with Complex Engineering Problem.

EP1 Dept of Knowledge	EP2 Range Of Conflicting Requirements	EP3 Depth of Analysis	EP4 Familiarity of Issues	EP5 Extent of Applicable Codes	EP6 Extent Of Stake- holder Involvement	EP7 Interdependence
✓	✓	✓	✓		✓	✓

5.4.1.1 Justifications for EP Attributes Mapping

EP1 – Depth of Knowledge:

- The project required knowledge of ensemble machine learning techniques such as Random Forest, XGBoost, CatBoost, LightGBM, and Logistic Regression.
- It required understanding of evaluation metrics including Precision, Recall, F1-score, and ROC-AUC.
- SHAP-based explainability required knowledge of feature attribution and Shapley values.
- Web security concepts such as session management and password hashing were also required.

EP2 – Range of Conflicting Requirements:

- A balance was required between model accuracy and interpretability.

- Complex ensemble models provided better performance but were more difficult to explain.
- Running multiple models increased prediction reliability but could affect system response time.

EP3 – Depth of Analysis:

- Model evaluation included Precision, Recall, F1-score, and ROC-AUC instead of accuracy alone.
- Special attention was given to the minority diabetic class due to dataset imbalance.
- A data-leakage issue in the dashboard accuracy calculation was identified and corrected using proper test-set accuracy.

EP4 – Familiarity of Issues:

- Diabetes prediction using machine learning is a well-studied research problem.
- However, integrating multi-model comparison, explainability, role-based access, and deployment created a more complex engineering challenge.
- The reviewed literature did not fully combine all these features in one system.

EP5 – Stakeholder Involvement:

- The system was designed for three stakeholder groups: patients, doctors, and administrators.
- Patients require a simple and fast prediction process, while doctors need transparency and prediction insights.
- Administrators require system-level control, including user and prediction management.

EP6 – Significant Consequences:

- Incorrect diabetes predictions may lead to unnecessary concern or delayed medical consultation.
- Therefore, model performance, prediction reliability, and responsible presentation of results were important considerations.
- The system was designed as a decision-support and risk prediction tool rather than a replacement for professional medical diagnosis.

EP7 – Interdependence:

- The machine learning models, preprocessing pipeline, backend, frontend, and dashboards are strongly interconnected.
- Training-time preprocessing objects such as LabelEncoder and StandardScaler must be consistently used during prediction.
- Any mismatch between components can affect the reliability of the overall system.

Mapping with Knowledge Profile

Table 5.2: Mapping with knowledge Profile.

K1 Natur al Scien ce	K2 Mathema tics	K3 Engineerin g Fundamen tals	K4 Speciali st Knowle dge	K5 Engineer ing Design	K6 Engineer ing Practice	K7 Comprehe nsion	K8 Resear ch Literatu re
		✓	✓	✓	✓		✓

5.4.1.1 Justifications for knowledge Profile Mapping (linked to EP1)

- KP3 – Core ML Concepts: Applied classification, train-test splitting, normalization, encoding, precision, recall, F1-score, and confusion matrices.
- KP4 – Specialist Knowledge: Studied Random Forest, XGBoost, LightGBM, class imbalance, and SHAP explainability.
- KP5 – System Design: Implemented Patient, Doctor, and Admin roles using Flask MVC and deployed all five models together.
- KP6 – Engineering Practice: Reused evaluate_model(), used Joblib for model persistence, and followed PEP 8 standards.
- KP8 – Research Literature: Used literature findings to select algorithms and address gaps in RBAC, multi-model comparison, and explainability.

5.4.2 Engineering Activities

Mapping with Complex Engineering Activities

The project lifecycle included, apart from the actual model construction, various engineering aspects including the sourcing and preparation of data, the design and construction of the deployed application as well as the test of its behaviour under realistic data usage. The following table describes these activities, with the engineering activity (EA) standard map to the right.

Table 5.3: Mapping with Complex Engineering Activities.

EA1 Range of re- sources	EA2 Level of Interaction	EA3 Innovation	EA4 Consequences for society and environment	EA5 Familiarity
✓	✓	✓	✓	✓

5.4.2.1 Justifications for Engineering Activities Mapping

EA1 – Range of Resources:

- The project utilized multiple technologies, including Python, Pandas, NumPy, Scikit-learn, XGBoost, LightGBM, and SHAP for data processing, modelling, and explainability.
- Flask, Chart.js, Font Awesome, and fpdf2 were used for web development, visualization, and report generation.
- Google Colab, Google Drive, and Ngrok supported model training, storage, and deployment.
- Integrating these diverse tools into a single functional application required significant technical coordination.

EA2 – Level of Interaction:

- The system supports three different user roles: Patient, Doctor, and Administrator.
- Each role interacts with different system features and interfaces based on specific requirements.
- Patients perform predictions, doctors access analytics and prediction logs, while administrators manage users and monitor the overall system.

EA3 – Innovation:

- The project combines multi-model comparative prediction, SHAP-based explainability, three-tier role-based access, and automated report generation.
- These features are integrated into a single deployed web-based system.
- The innovation lies in combining existing technologies into a unified and practical diabetes prediction platform rather than developing a completely new algorithm.

EA4 – Consequences:

- The system has potential value in supporting early diabetes risk screening, particularly in resource-limited areas of Bangladesh.
- Early risk identification may encourage users to seek professional medical consultation.
- The use of relatively lightweight machine learning models reduces computational requirements compared to complex deep-learning approaches.

EA5 – Familiarity:

- The team had prior experience with Flask and core machine learning frameworks such as Scikit-learn and XGBoost.
- This allowed the team to focus on more challenging aspects such as system integration, role-based access control, explainability, and deployment.
- Therefore, the project involved both familiar technologies and new integration challenges.

5.5 Summary

While not modeling the project and not covering implementation issues, as covered in the previous part of the report, this chapter explored the engineering standards, the ethics of the project, the financial considerations, and the structural complexity of the project. The choice of the standards side was similar, with Flask preferred as the classical machine-learning stack, REST-based HTTP/JSON communication over alternatives like django, deep learning and WebSockets, both times for reasons related to the actual constraints of the project - the small team, the tight schedule and the tabular data that uses no heavier solution. The "social and environmental footprint" piece was non-marketing - the system does have significant utility in helping to interpret the lab data and assist in the early screening, but it is not intended or meant to replace the underlying lab tests, and that it has some significant security and remarkability issues was mentioned without mincing words. The financial analysis revealed that a totally free and open source tool suite was used for creating Phase-I at a cost of about BDT 2,900 (free and open without any costs at all), and a clearly larger budget (BDT 5000-9000 per month) was needed to progress towards a persistent, production grade deployment. To wrap this up, the project was mapped to both Complex Engineering Problem and Engineering Activity, revealing that the complexity is less related to any single algorithm, and more related to its interdependence between the five models, the three user roles and a live deployment pipeline to make everything work as expected, encompassing the kind of integrated engineering challenge these two frameworks are supposed to capture.

Chapter 6

Conclusion

The end of the chapter pulls all the elements in and wraps them up and recognizes that there are elements that are still lacking in the current system, but provides a plausible next set of steps to make the process an actual "system" that a clinic might serve.

6.1 Summary

This was a project initiated to create a system for predicting risk of diabetes that didn't sit in a notebook with different algorithms trained inside, but was something the patient, the doctor, or the administrator could open and use. That is one of the reasons why in earlier chapters we could see so many choices being made: training 5 classifiers together instead of selecting just one, having each classifier be presented in a different Flask application and each one have 3 different kinds of users, as well as presenting every prediction with a SHAP explanation instead of a label alone. Model-wise, the five models were trained on 80,000 records and tested on 20,000 held-out records and LightGBM outperformed XGBoost, who had 97.13% accuracy, and Random Forest who had 97.05% accuracy, on the metrics most important for a screening tool: on the diabetic class, LightGBM achieved an accuracy of 97.24% and F1 score of 81.04%, whereas Random Forest scored 97% accuracy and XGBoost 97% accuracy. The more permanent benefit of the project will be the app itself, rather than the statistics. A patient can enter their 8 usual clinical values and get predictions from the 5 models simultaneously, the ensemble risk score along with a simple language recommendation, a SHAP breakdown of which of their own values determined the result to go one way or the other and a PDF to bring to a real appointment. Both doctors and administrators have different perspectives on the same data with statistics, model comparisons and a log of all predictions made on the platform. Again, as mentioned directly in Chapter 4: An inconsistency was discovered in calculations of the "best model" figure during the admin dashboard's construction: it was computing the accuracy on the training, rather than testing, set, therefore giving an artificial sense of the Random Forest's accuracy. The development of the prediction pipeline is an outcome of this project, and so is catching and correcting for that in how the results are read - not letting the inflated number pass. Overall, the system demonstrates that the implementation of a system capable of supporting diabetes screening from multiple models with explainability and considered a role-based system is both theoretically desirable and feasible for development.

6.2 Limitation

Although the findings of this research are promising, the limitations that are likely to be considered are as follows:

- **Limited Feature Set:** The system utilizes eight accessible clinical and demographic features. Risk factors such as family history, physical activity and diet, were not incorporated.
- **Dataset Generalizability:** The dataset was obtained from a public Kaggle repository, rather than a Bangladeshi clinical population.
- **Development Server:** The application utilizes insecure built-in development Flask server with Ngrok, which is inappropriate for production-grade software.
- **Security Limitations:** The system utilizes only basic password hashing, rather than more secure bcrypt/Argon2. The system does not implement CSRF protection or application-layer HTTPS.

6.3 Future Work

- **Feature Expansion:** Future work could expand the feature set to include not only BMI, but also family history, waist-hip ratio, physical activity and diet.
- **Bangladesh-Specific Validation:** Conduct clinical validation to evaluate the algorithm's performance on a truly Bangladeshi population.
- **Production Deployment:** Replace the built-in Flask development server and Ngrok with a production-grade WSGI server such as Gunicorn, deployed behind Nginx with HTTPS and appropriate security configurations.
- **Strengthen Security:** Future work should add more robust security measures such as bcrypt/Argon2 password hashing, CSRF protection, HTTPS and production-grade server deployment.
- **Conduct User Acceptance Testing:** Future work should conduct formal user acceptance testing with doctors and prospective patients, and incorporate their feedback into the design process.

References

- [1] International Diabetes Federation, "IDF Diabetes Atlas," 10th ed. Brussels, Belgium: IDF, 2021. [Online]. Available: <https://www.diabetesatlas.org>
- [2] World Health Organization, "Diabetes," WHO Fact Sheets, 2023. [Online]. Available: <https://www.who.int/news-room/fact-sheets/detail/diabetes>
- [3] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.
- [4] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, San Francisco, CA, USA, 2016, pp. 785–794.
- [5] G. Ke *et al.*, "LightGBM: A highly efficient gradient boosting decision tree," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 30, 2017, pp. 3146–3154.
- [6] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, 2001.
- [7] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *J. Artif. Intell. Res.*, vol. 16, pp. 321–357, 2002.
- [8] H. He, Y. Bai, E. A. Garcia, and S. Li, "ADASYN: Adaptive synthetic sampling approach for imbalanced learning," in *Proc. IEEE World Congr. Comput. Intell. (IJCNN)*, 2008, pp. 1322–1328.
- [9] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 30, 2017, pp. 4765–4774.
- [10] S. M. Lundberg *et al.*, "From local explanations to global understanding with explainable AI for trees," *Nat. Mach. Intell.*, vol. 2, no. 1, pp. 56–67, 2020.
- [11] M. T. Ribeiro, S. Singh, and C. Guestrin, "'Why should I trust you?': Explaining the predictions of any classifier," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, 2016, pp. 1135–1144.
- [12] S. Ahmed, M. S. Kaiser, M. S. Hossain, and K. Andersson, "A comparative analysis of LIME and SHAP interpreters with explainable ML-based diabetes predictions," *IEEE Access*, vol. 13, pp. 37370–37388, 2024, doi: 10.1109/ACCESS.2024.3374285.
- [13] I. Tasin, T. U. Nabil, S. Islam, and R. Khan, "Diabetes prediction using machine learning and explainable AI techniques," *Healthc. Technol. Lett.*, vol. 10, no. 1–2, pp. 1–10, 2023, doi: 10.1049/htl2.12039.
- [14] M. K. Hasan, M. A. Alam, D. Das, E. Hossain, and M. Hasan, "Diabetes prediction using ensembling of different machine learning classifiers," *IEEE Access*, vol. 8, pp. 76516–76531, 2020.
- [15] R. Hasan, V. Dattana, S. Mahmood, and S. Hussain, "Towards transparent diabetes prediction: Combining AutoML and explainable AI for improved clinical insights," *Information*, vol. 16, no. 1, p. 7, 2024, doi: 10.3390/info16010007.
- [16] P. B. Khokhar *et al.*, "Towards transparent and accurate diabetes prediction using machine learning and explainable artificial intelligence," arXiv preprint arXiv:2501.18071, 2025.

- [17] F. Rustam *et al.*, "Enhanced detection of diabetes mellitus using novel ensemble feature engineering approach and machine learning model," *Sci. Rep.*, vol. 14, p. 23274, 2024, doi: 10.1038/s41598-024-74357-w.
- [18] M. M. Chowdhury, R. S. Ayon, and M. S. Hossain, "An investigation of machine learning algorithms and data augmentation techniques for diabetes diagnosis using class imbalanced BRFSS dataset," *Healthc. Anal.*, vol. 5, p. 100297, 2024, doi: 10.1016/j.health.2023.100297.
- [19] K. Amara *et al.*, "Interpretable machine learning framework for diabetes prediction: Integrating SMOTE balancing with SHAP explainability for clinical decision support," *Healthcare*, vol. 13, no. 20, p. 2588, 2025, doi: 10.3390/healthcare13202588.
- [20] M. Kiran *et al.*, "Machine learning and artificial intelligence in type 2 diabetes prediction: A comprehensive 33-year bibliometric and literature analysis," *Front. Digit. Health*, vol. 7, p. 1557467, 2025, doi: 10.3389/fdgth.2025.1557467.
- [21] N. P. Tigga and S. Garg, "Prediction of type 2 diabetes using machine learning classification methods," *Procedia Comput. Sci.*, vol. 167, pp. 706–716, 2020.
- [22] D. Sisodia and D. S. Sisodia, "Prediction of diabetes using classification algorithms," *Procedia Comput. Sci.*, vol. 132, pp. 1578–1585, 2018.
- [23] N. Yuvaraj and S. R. VijaySai, "Diabetes prediction using machine learning algorithms with feature selection and dimensionality reduction," *Mater. Today Proc.*, vol. 45, pp. 1701–1707, 2021.
- [24] I. Kavakiotis, O. Tsave, A. Salifoglou, N. Maglaveras, I. Vlahavas, and I. Chouvarda, "Machine learning and data mining methods in diabetes research," *Comput. Struct. Biotechnol. J.*, vol. 15, pp. 104–116, 2017.
- [25] M. Maniruzzaman *et al.*, "Accurate diabetes risk stratification using machine learning: Role of missing value and outliers," *J. Med. Syst.*, vol. 42, no. 5, p. 92, 2018.
- [26] M. Alehegn, R. Joshi, and P. Mulay, "Analysis and prediction of diabetes diseases using machine learning algorithm: Ensemble approach," *Int. Res. J. Eng. Technol.*, vol. 6, no. 5, pp. 1–8, 2019.
- [27] G. Battineni, G. G. Sagaro, N. Chintalapudi, and F. Amenta, "Applications of machine learning predictive models in the chronic disease diagnosis," *J. Pers. Med.*, vol. 10, no. 2, p. 21, 2020.
- [28] V. Chang, J. Bailey, Q. A. Xu, and Z. Sun, "Pima Indians diabetes mellitus classification based on machine learning (ML) algorithms," *Neural Comput. Appl.*, vol. 35, no. 22, pp. 16157–16173, 2023.
- [29] J. Chaki, S. T. Ganesh, S. K. Cidham, and S. A. Theertan, "Machine learning and artificial intelligence based diabetes mellitus detection and self-management: A systematic review," *J. King Saud Univ.–Comput. Inf. Sci.*, vol. 34, no. 6, pp. 3204–3225, 2022.
- [30] U. Das and B. Ahmed, "An explainable ML approach for diabetes detection using the influential features," in *Proc. 26th Int. Conf. Comput. Inf. Technol. (ICCIIT)*, 2023, pp. 1–5.
- [31] G. Dharmarathne, T. N. Jayasinghe, M. Bogahawaththa, D. Meddage, and U.

- Rathnayake, "A novel machine learning approach for diagnosing diabetes with a self-explainable interface," *Healthc. Anal.*, vol. 5, p. 100301, 2024.
- [32] G. Ekambaram, T. Rakkiannan, N. Palanisamy *et al.*, "A novel method for early detection and prediction of diabetes using various machine learning and deep learning models," *Int. J. Diabetes Dev. Ctries.*, 2025, doi: 10.1007/s13410-025-01590-y.
- [33] Q. Sun, X. Cheng, K. Han *et al.*, "Machine learning-based assessment of diabetes risk," *Appl. Intell.*, vol. 55, p. 106, 2025, doi: 10.1007/s10489-024-05912-1.
- [34] N. Das, J. Dutta, A. Bhattacharyya, and S. Raj, "Optimizing early diabetes detection through machine learning: A comparative analysis of classification models," in *Intell. Vis. Comput. (ICIVC 2024)*, Lect. Notes Smart Technol., Singapore: Springer, 2025, doi: 10.1007/978-981-96-4722-4_19.
- [35] V. G. Narendra and G. Shivaprasad, "Web application for diabetes prediction using machine learning techniques," *WSEAS Trans. Comput.*, vol. 23, pp. 465–479, 2024.
- [36] H. Lee *et al.*, "Prediction model for type 2 diabetes mellitus and its association with mortality using machine learning in three independent cohorts from South Korea, Japan, and the UK," *eClinicalMedicine*, vol. 80, p. 103069, 2025, doi: 10.1016/j.eclinm.2025.103069.
- [37] M. D. Hasan, J. Li *et al.*, "Privacy-preserving collaborative diabetes prediction in heterogeneous health care systems: Algorithm development and validation of a secure federated ensemble framework," *JMIR Diabetes*, 2026, doi: 10.2196/79166.
- [38] S. Manjrekar, N. Ghongade, S. Ajankar, and A. Pai, "A comprehensive ensemble machine learning framework for robust diabetes prediction," in *Intell. Vis. Comput. (ICIVC 2025)*, Lect. Notes Netw. Syst., vol. 1712, Springer, 2026, doi: 10.1007/978-3-032-10670-4_29.
- [39] The Diabetes Prevention Program Research Group, "Reduction in the incidence of type 2 diabetes with lifestyle intervention or metformin," *N. Engl. J. Med.*, vol. 346, no. 6, pp. 393–403, 2002.
- [40] C. Molnar, *Interpretable Machine Learning: A Guide for Making Black Box Models Explainable*, 2nd ed. Leanpub, 2022. [Online]. Available: <https://christophm.github.io/interpretable-ml-book/>